Accelerated Computing

Motion Estimation:

Block Matching

Development and analysis of CUDA solutions for the block matching problem.

Introduction

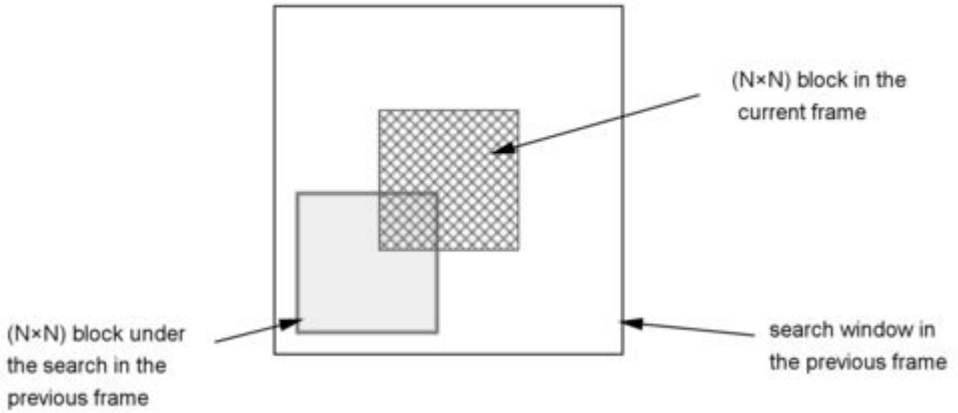
Introduction to block matching

Block Matching

Motion Estimation is a fundamental technique in video processing for tracking the movement of objects between consecutive frames

Block Matching is the standard and most commonly used algorithm to implement this technique. It works by dividing the current frame into blocks and searching for the most similar block in the reference frame (previous one) using Sum of Absolute Difference (SAD) metric.

Once the match is found, it is possible to generate the **Motion Vector**, a vector that describes the spatial displacement of the block occurring between the two frames.



Use case

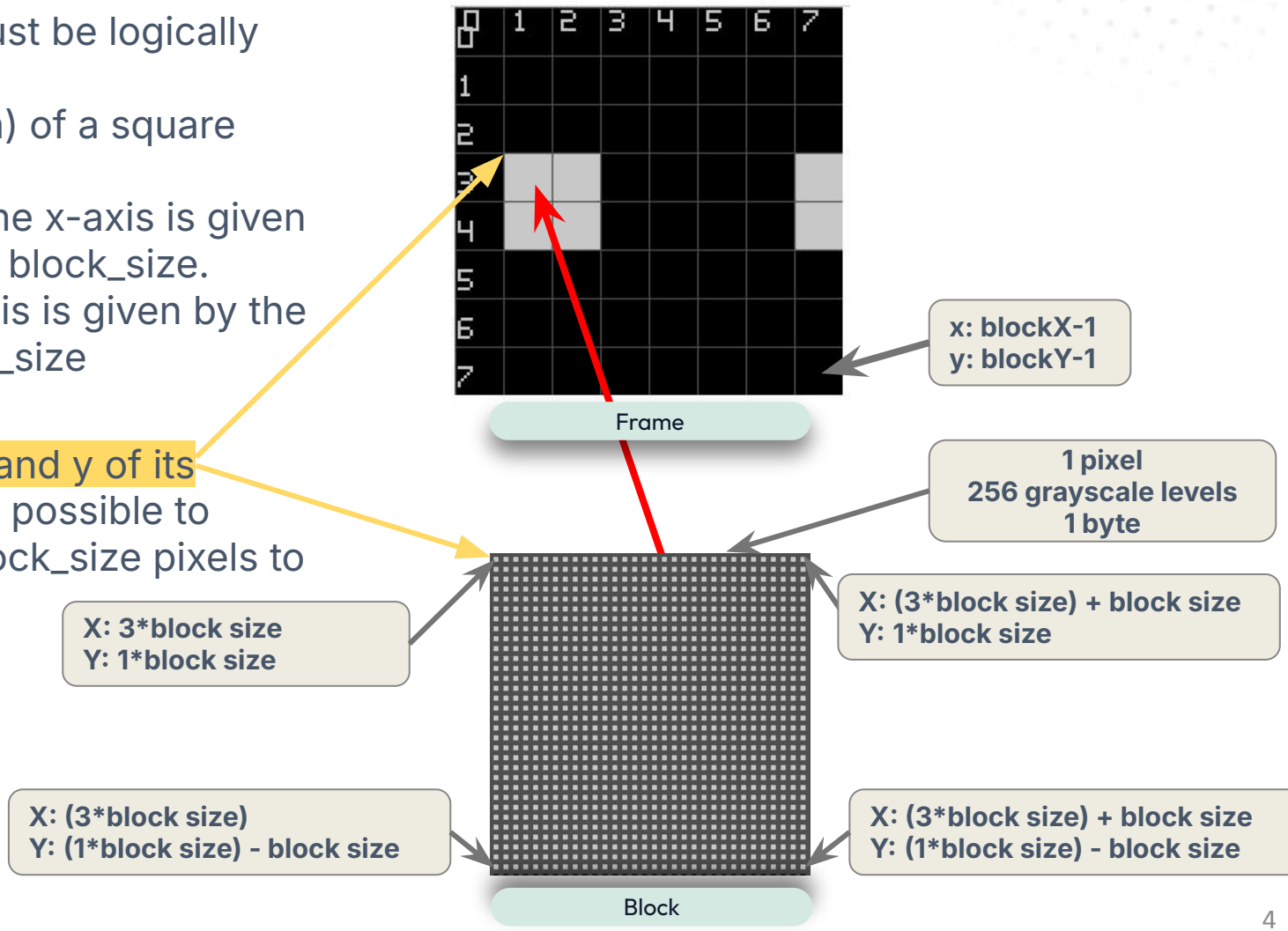
In video compression (H.264), instead of saving the entire image pixel by pixel for each frame, only the motion vector and the residual error (encoded via the Sum of Absolute Difference)

Frame grid

The reference frame and current frame must be logically divided into blocks:

1. **Block size**: the dimension (side length) of a square block.
2. **BlockX**: the number of blocks along the x-axis is given by the frame width (pixels) divided by block_size.
3. **BlockY**: the number of blocks on y-axis is given by the frame height (pixels) divided by block_size

A block is identified by the coordinates x and y of its top-left pixel. Starting from that point, it is possible to move within the square block for up to block_size pixels to the right and downward.



Sum Absolute Difference (SAD)

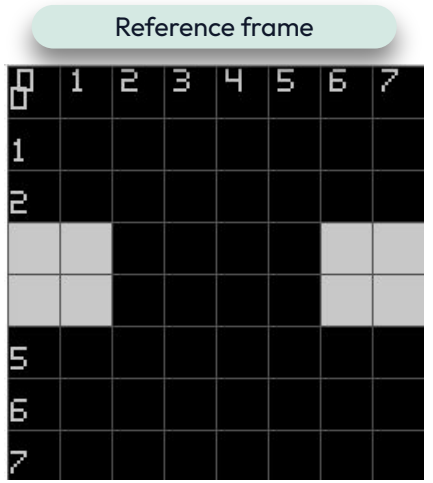
To find the differences between two blocks, the Sum Absolute Differences (SAD) is used, which calculates the sum of the differences between each pixel of the block in the current frame and the corresponding pixel in the reference frame.

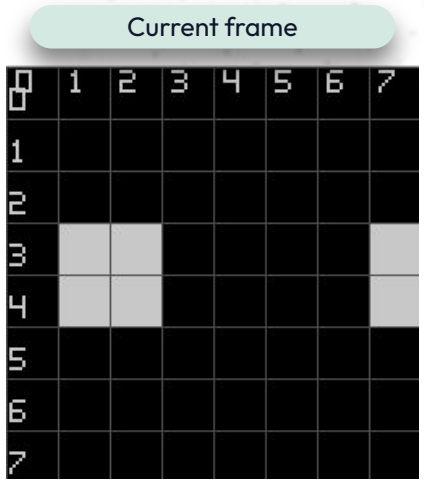
The block with the smallest SAD is considered the best match.

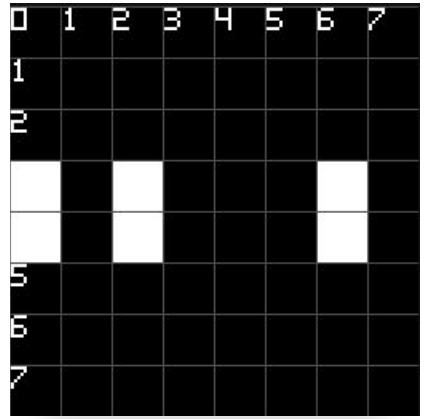
$$SAD(dx, dy) = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} |I_{curr}(x+i, y+j) - I_{ref}(x+dx+i, y+dy+j)|$$

SAD.cpp

```cpp
static inline int computeSAD(const ImageGray& curr, const ImageGray& ref,
 int x1, int y1, int x2, int y2, int blockSize) {
  int sad = 0;
  for(int y = 0; y < blockSize; y++) {
   for(int x = 0; x < blockSize; x++) {
    sad += abs(curr.at(x1 + x, y1 + y) - ref.at(x2 + x, y2 + y));
   }
  }
  return sad;
}
```







Motion Vector (MV)

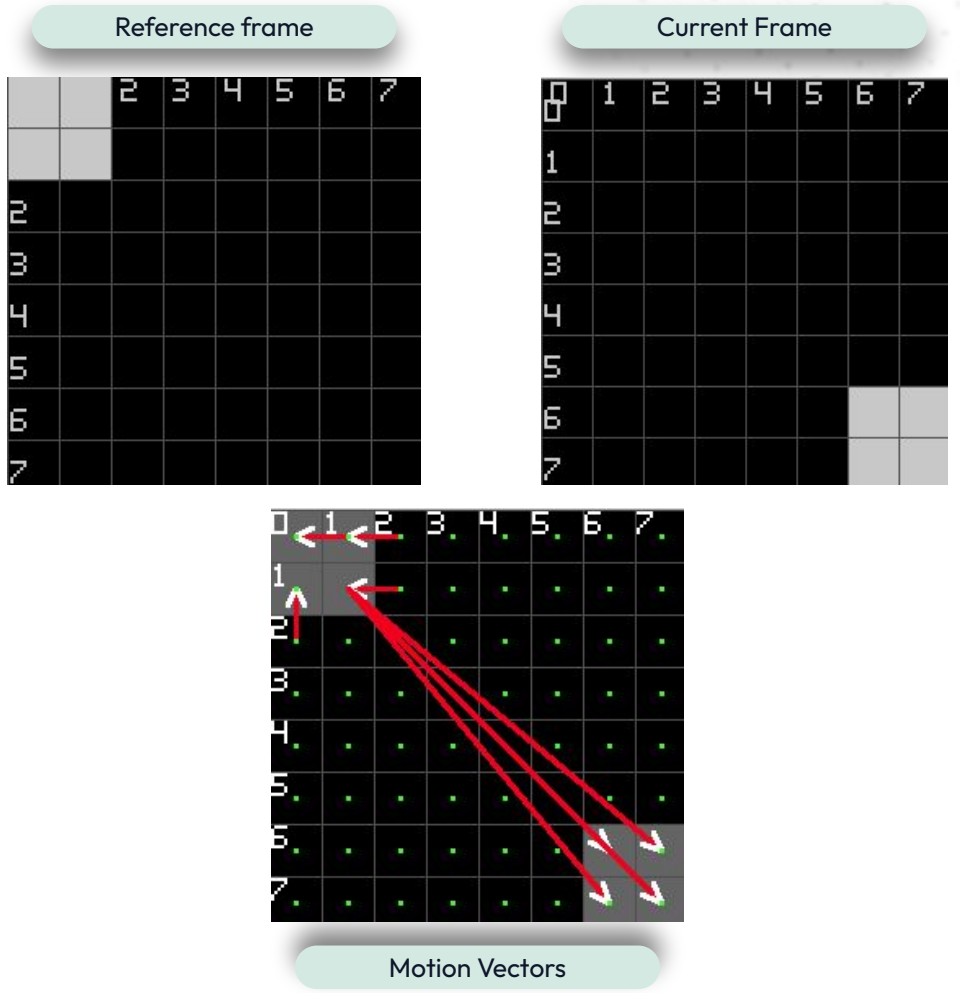
Motion Vector describes the spatial displacement of a block between a reference frame and the current frame.

Only the displacement coordinates (dx, dy) are stored.

$$[dx, dy] = reference\_coordinates - current\_coordinates$$

If a current frame block matches multiple reference blocks with the same SAD value, the shortest motion vector is chosen (i.e., the closest block).

Current Frame	Coordinate Shift
[0,0]	[2,0]
[0,1]	[1,0]
[1,0]	[0,1]
[1,1]	[1,0]
[6,6]	[-5,-5]
[6,7]	[-5,-6]



Available strategies

Choosing the subset of blocks in the reference frame to compare with the current frame defines the different Block Matching strategies, balancing accuracy and computational cost.

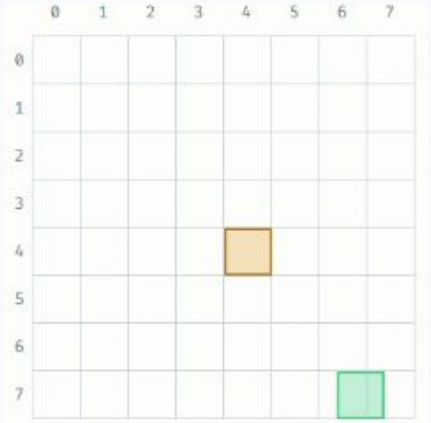
Full Search

Every block of the current frame is compared with all blocks present in the entire reference frame. It guarantees the best possible match but has a high computational cost.

Available strategies

Choosing the subset of blocks in the reference frame to compare with the current frame defines the different Block Matching strategies, balancing accuracy and computational cost.

Full Search



$O(N^2)$

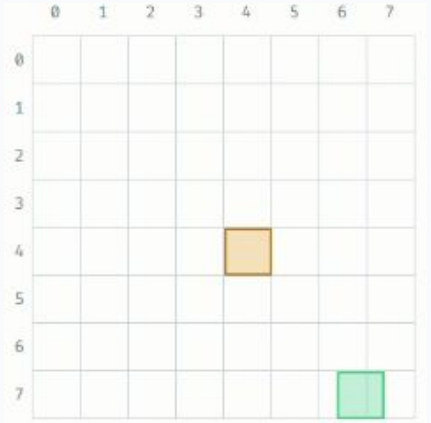
Range Search

Each block in the current frame is compared only with the blocks within a restricted search area defined by the search range.

Available strategies

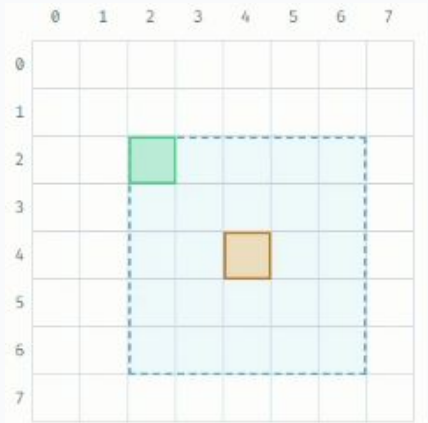
Choosing the subset of blocks in the reference frame to compare with the current frame defines the different Block Matching strategies, balancing accuracy and computational cost.

Full Search



$O(N^2)$

Range Search



$O(N \cdot R)$

Logarithmic search

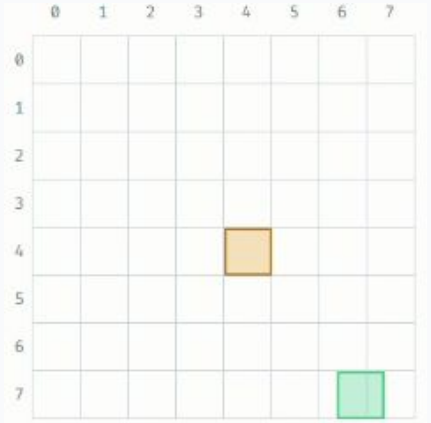
Given a distance d, the block is compared with 9 grid points: the center (x,y) and the 8 directions at distance d (x±d, y±d), (x±d, y), (x, y±d).

The process iterates, halving d until d=1 (unless a zero-SAD value is found earlier), restarting each time from the block with the smallest SAD found in the previous step.

Available strategies

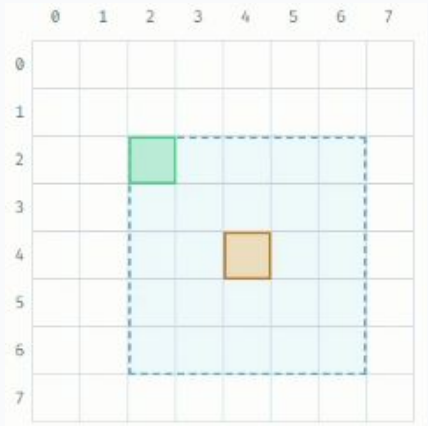
Choosing the subset of blocks in the reference frame to compare with the current frame defines the different Block Matching strategies, balancing accuracy and computational cost.

Full Search



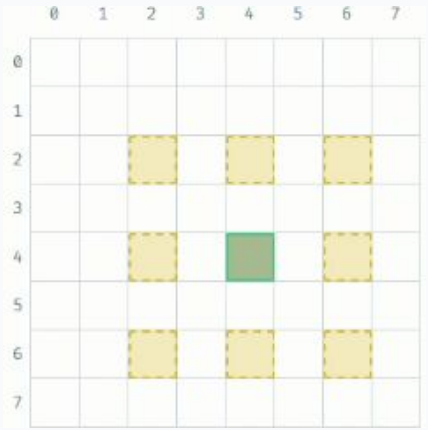
$O(N^2)$

Range Search



$O(N \cdot R)$

Logarithmic search



$O(N \cdot \log D)$

Full Search

Implementations of full search algorithm

CPU Naive

Sequential implementation:

1. Logically divide the current frame into blocks.
2. For each block, compute the SAD with every candidate block in the reference frame.
3. Select the candidate with the lowest SAD (ties broken by the shortest motion vector).
4. Repeat for every block in the current frame.

fullSearchBM_cpu_naive.cpp

```cpp
// Iterate over every block in the current frame
for (int by = 0; by < blocksY; by++) {
 for (int bx = 0; bx < blocksX; bx++) {
  int bestSAD = INT_MAX; MotionVector bestMV{0, 0};
  // Scan every candidate block in the reference frame (full search)
  // No RAW dependency: candidates are independent
  for (int refY = 0; refY < blocksY; refY++) {
   for (int refX = 0; refX < blocksX; refX++) {
    int sad = computeSAD(curr, ref, bx, by, refX, refY);
    // tie-break on distance: choose the spatially closest candidate
    int dist = (refX - bx) * (refX - bx) + (refY - by) * (refY - by);
    int bestDist = bestMV.dx * bestMV.dx + bestMV.dy * bestMV.dy;
    if (sad < bestSAD || (sad == bestSAD && dist < bestDist)) {
     bestSAD = sad;
     bestMV = {refX - bx, refY - by};
    }
   }
  }
  mv[bx][by] = bestMV; // store the optimal motion vector
 }
}
```

⚠️ **Parallelization & Execution Dependency**

Finding the motion vector for different blocks of the current frame is completely independent. However, this CPU implementation processes them sequentially, without exploiting this parallelism.

CUDA Naive

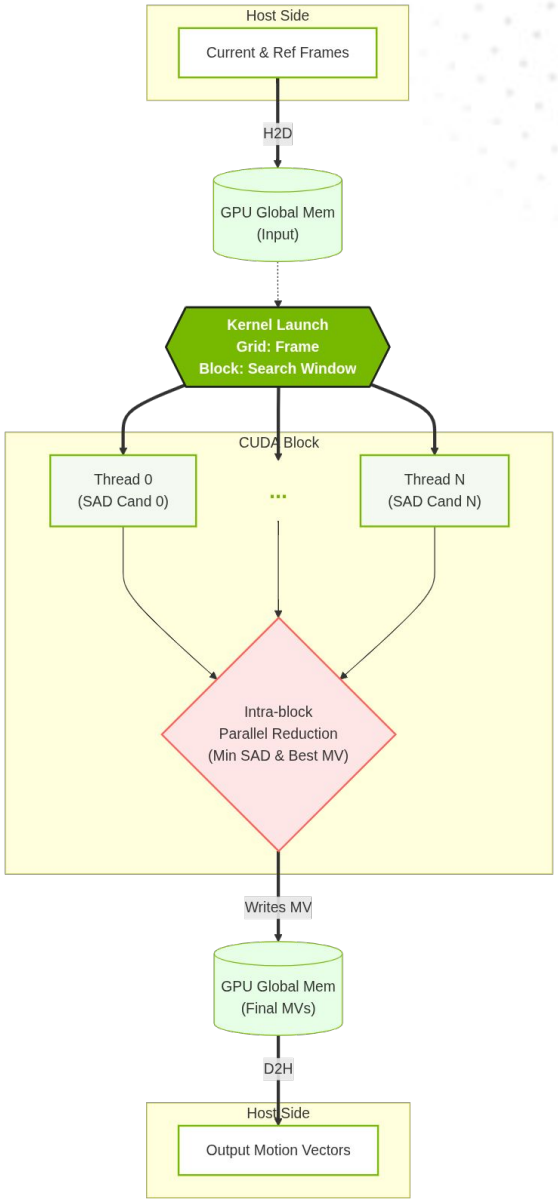
Instead of processing the current frame block by block, parallelize the computation to find the best SAD value.

The idea is to map the search problem onto the GPU's architecture:

- **CUDA grid 2D**: maps the frame grid. Each CUDA Block processes exactly one block of the current frame.
- **CUDA block 2D:** maps the search window (frame grid). Each thread calculates the SAD for at least one candidate reference frame block.

Workflow idea:

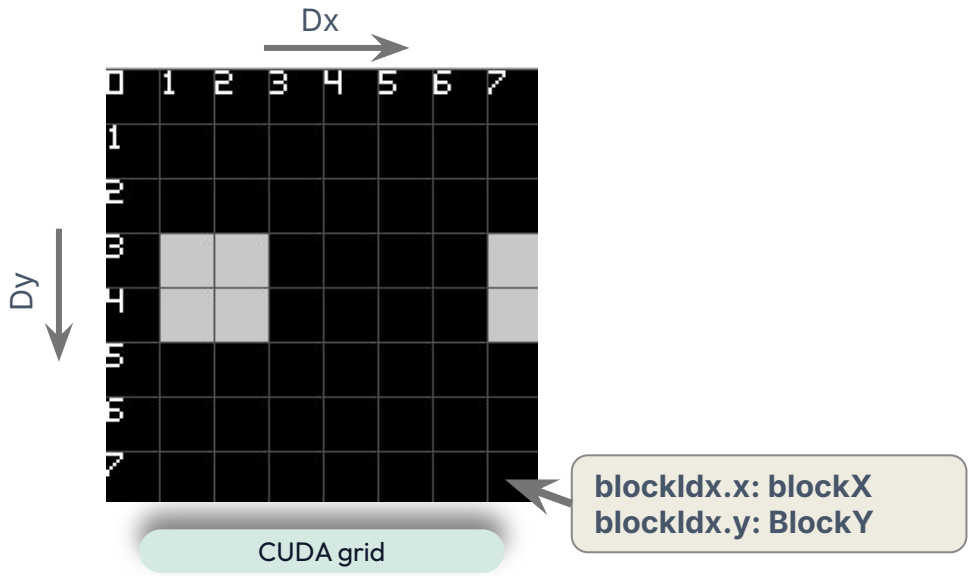
1. **Host Side**: transfer (H2D) the current and reference frames into GMEM.
2. **Kernel Side:**
   a. **Candidates Evaluation**: threads compute the SAD in parallel for all search positions (reference frame blocks).
   b. **Block-Wide Reduction**: parallel reduction within the block to find the absolute minimum SAD (and shortest MV) among all candidates. The final result is saved to GMEM.
3. **Host Side:** transfer (D2H) the final MVs result back to the host.



CUDA Naive: grid and block dimensions

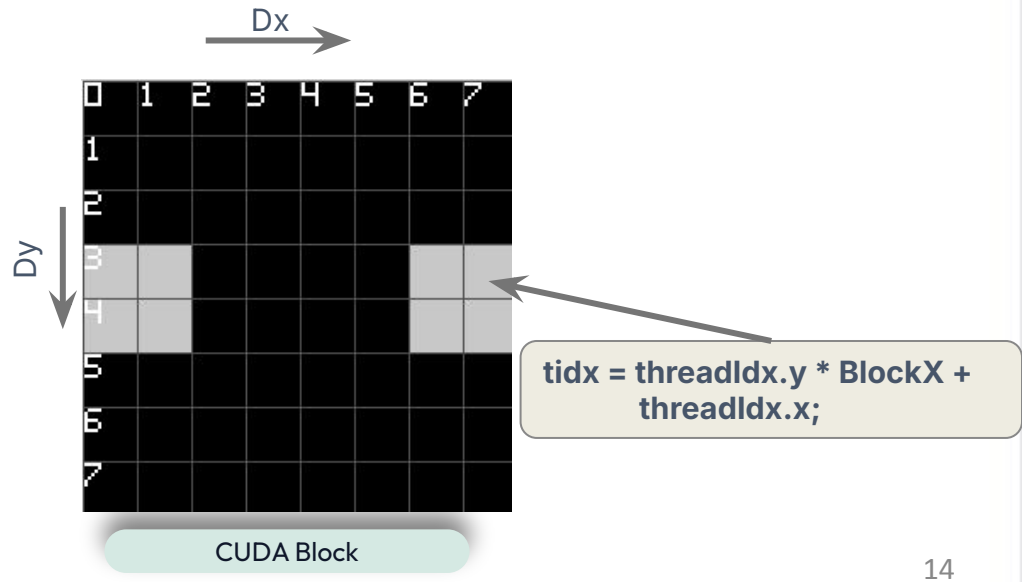
GridDim: 2 dimensions [blocksX, blocksY]

- The CUDA grid has the same dimensions as the frame grid.
- Each CUDA block is responsible for matching its corresponding block in the current frame against the reference frame.



BlockDim: 2 dimensions [blocksX, blocksY]:

- The CUDA block conceptually maps the search area, but its dimensions are strictly forced to be a square.
- Each CUDA thread computes the SAD value for one or more specifically assigned candidate blocks in the reference frame.
- The total number of threads must be a power of 2 and the total threads must not exceed the hardware limit.



CUDA Naive: kernel (Candidates Evaluation)

$$CandidateReferenceBlocks \leq threads_{blocks}$$

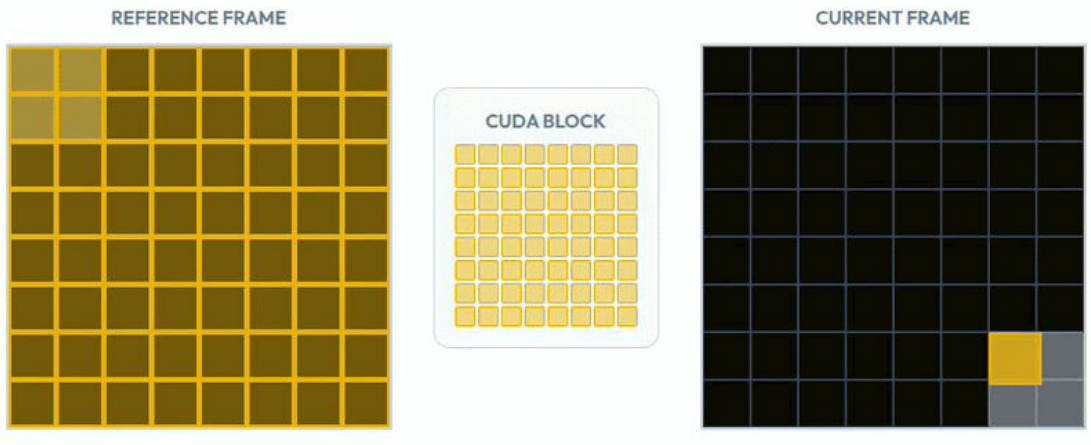
1. Sequentially computes the SAD value between the current frame block and a specific candidate block within the reference frame, comparing pixel by pixel.

fullSearchBM_cuda_naive.cpp: fullSearchKernel

```
// Convert linear thread index to 2D block coordinates
ref_bx = tidx % blockX; ref_by = tidx / BlockY;
// Convert reference block coordinates to pixel coordinate
ref_x = ref_bx * blockSize; ref_y = ref_by * blockSize;
// Compute SAD for this candidate
sad = computeSAD_device(curr, ref, curr_x, curr_y, ref_x, ref_y, blockSize, w,h);
// Calculate motion vector and distance for tie-breaking
dx = ref_bx - curr_bx; dy = ref_by - curr_by; distance = dx*dx + dy*dy;
```

fullSearchBM_cuda_naive.cpp: computeSAD_device

```
static __device__ __inline__ int computeSAD_device(const unsigned char* curr,
 const unsigned char* ref, int curr_x, int curr_y, int ref_x, int ref_y,
 int blockSize, int w, int h) {
  int sad = 0;
  // Iterate over block rows
  for(int by = 0; by < blockSize; by++) {
   // Calculate current pixel row of current and reference frame block
   int cy = curr_y + by; int ry = ref_y + by;
   // Check if rows exceed frame boundaries, return invalid SAD if out frame
   // Iterate over block columns
   for(int bx = 0; bx < blockSize; bx++) {
    // Calculate current pixel row of current and reference frame block
    int cx = curr_x + bx:int rx = ref_x + bx;
    // Check if columns exceed frame boundaries, return invalid SAD if out of frame
    // Accumulate SAD
    sad = __usad(curr[cy * w + cx], ref[ry * w + rx], sad);
   }
  }
  return sad;
}
```



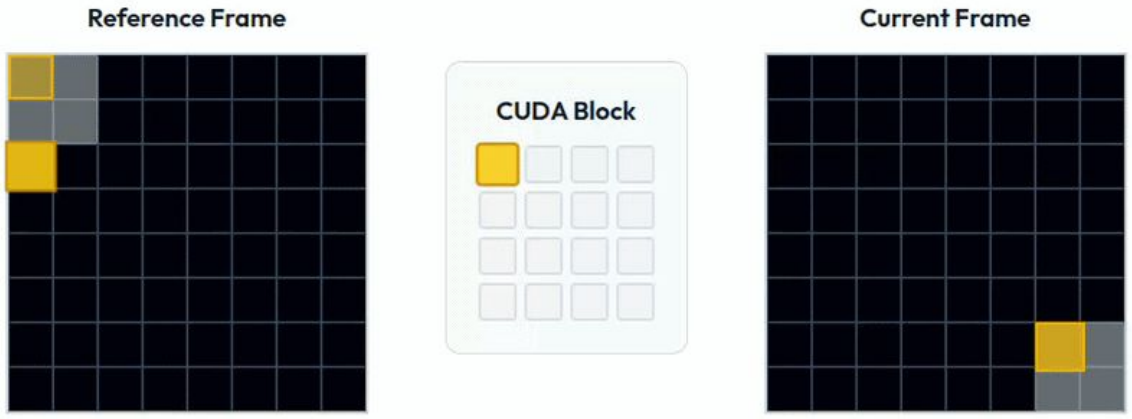
CUDA Naive: kernel (Candidates Evaluation)

$$CandidateReferenceBlocks \leq threads_{blocks}$$

1. Sequentially computes the SAD value between the current frame block and a specific candidate block within the reference frame, comparing pixel by pixel.

$$CandidateReferenceBlocks > threads_{blocks}$$

1. Computes SAD values for multiple candidate positions using a thread-coarsening strategy, where each thread iterates over candidate positions assigned through a block-strided loop.
2. Stores the minimum local SAD value, along with the corresponding MV and distance, to build the final result



fullSearchBM_cuda_naive.cpp: fullSearchKernel

```
// Each thread processes multiple candidate positions with stride = total_threads
for (pos = tidx; pos < total_candidates; pos += total_threads) {
 // Convert linear index to 2D block coordinates
 ref_bx = pos % width; ref_by = pos / width;
 // Convert reference block coordinates to pixel coordinate
 ref_x = ref_bx * blockSize; ref_y = ref_by * blockSize;
 // Compute SAD for this candidate
 sad = computeSAD_device(curr, ref, curr_x, curr_y, ref_x, ref_y, blockSize, w,h);
 // Calculate motion vector and distance for tie-breaking
 dx = ref_bx - curr_bx; dy = ref_by - curr_by; distance = dx*dx + dy*dy;
 // Update local_best match for the thread
}
```

fullSearchBM_cuda_naive.cpp: computeSAD_device

```
static __device__ __inline__ int computeSAD_device(const unsigned char* curr,
 const unsigned char* ref, int curr_x, int curr_y, int ref_x, int ref_y,
 int blockSize, int w, int h) {
  int sad = 0;
  // Iterate over block rows
  for(int by = 0; by < blockSize; by++) {
   // Calculate current pixel row of current and reference frame block
   int cy = curr_y + by; int ry = ref_y + by;
   // Check if rows exceed frame boundaries, return invalid SAD if out frame
   // Iterate over block columns
   for(int bx = 0; bx < blockSize; bx++) {
    // Calculate current pixel row of current and reference frame block
    int cx = curr_x + bx; int rx = ref_x + bx;
    // Check if columns exceed frame boundaries, return invalid SAD if out of frame
    // Accumulate SAD
    sad = __usad(curr[cy * w + cx], ref[ry * w + rx], sad);
   }
  }
  return sad;
}
```

CUDA Naive: kernel (Block-Wide Reduction)

Following the computation of local best results, a block-wide reduction is required to determine the optimal match for the current frame block using *cub::BlockReduce*:

1. *blockReduceT*: type definition of the block reduction algorithm.
2. *temp_storage*: shared memory explicitly allocated, with size and layout determined by CUB at compile-time.
3. *block_best*: parallel aggregation of all *local_best* variables across threads, using the custom *CandidateSadOp* comparator, which selects the candidate with the lowest SAD, breaking ties first by shortest motion vector, then by a deterministic ordering rule.
4. Result Storage: thread 0 writes the final motion vector to GMEM.

fullSearchBM_cuda_naive.cpp: fullSearchKernel

```
// Block reduction to find the best match among all threads in the block
typedef cub::BlockReduce<CandidateSad, threadsPerBlockDim, cub::BLOCK_REDUCE_WARP_REDUCTIONS,
 threadsPerBlockDim> blockReduceT;

// Allocate shared memory for reduction (size/layout determined by CUB at compile-time)
__shared__ typename blockReduceT::TempStorage temp_storage;

// Aggregate local_best from all threads using custom CandidateSadOp comparator
CandidateSad block_best =
 blockReduceT(temp_storage).Reduce(local_best, MotionVectorUtils::CandidateSadOp());

// Thread 0 writes the optimal motion vector to global memory
if (tidx == 0) {
    d_mv[blockIdx.y * gridDim.x + blockIdx.x] = {block_best.dx, block_best.dy};
}
```

cuda_utils.h: CandidateSadOp

```
__device__ __forceinline__ CandidateSad operator()(
 const CandidateSad& current, const CandidateSad& best) const {
  // Primary: lowest SAD
  if (current.sad != best.sad) return current.sad < best.sad ? current : best;
  // Tie-break 1: shortest motion vector (Euclidean distance)
  int dist_current = current.dx * current.dx + current.dy * current.dy;
  int dist_best = best.dx * best.dx + best.dy * best.dy;
  if (dist_current != dist_best)
   return dist_current < dist_best ? current : best;
  // Tie-break 2: deterministic ordering (smaller dy, then smaller dx)
  return (current.dy != best.dy) ?
   (current.dy < best.dy ? current : best) :
   (current.dx < best.dx ? current : best);
}
```

CUDA Naive: profiling

Profiling SAD computation with 1024×1024 and block size of 32 (1024 pixels), so frame grid has 32×32 frame blocks

- **Frame size:** 1024 KiB
- **CUDA grid**: 32×32×1 blocks.
- **CUDA block:** 32×32×1 threads.

Profiling

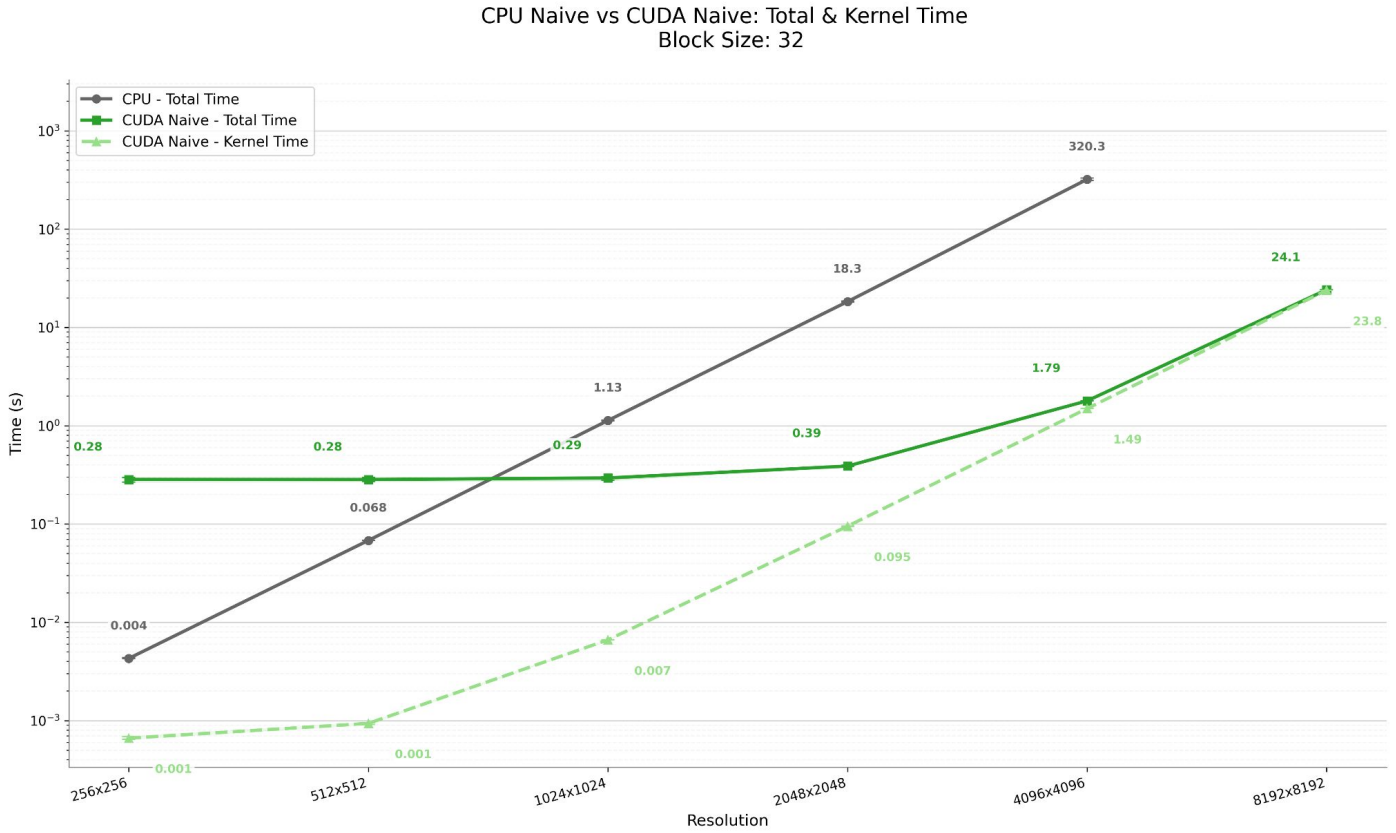
- Compute (SM) throughput: 71.19% & Memory Throughput: 95.79%: memory-bound.
- L1 Cache Throughput: 99.57% & L1 Cache Hit Rate: 97.00%:
  - L1 cache completely saturated. Non-coalesced accesses fragment requests.
- L2 Cache Throughput: 6.23% & L2 Cache Hit Rate: 99.68%:
  - L1 cache absorbs nearly all traffic before reaching L2.
- Uncoalesced Accesses, 94% waste:
  - Non-coalesced accesses force global memory to fetch a full 128-byte segment per request, even though each thread only uses 4-8 bytes.
- Achieved Occupancy 97.45%:
  - Due only to CUB SMEM overhead and low register pressure.

CUDA Naive: benchmark

GMEM Inefficiency: non-coalesced memory access pattern generates excessive memory traffic, saturating bandwidth.

H2D Bottleneck: memory transfers dominate execution time at lower resolutions. The kernel's computational cost emerges and saturates GPU resources only on extreme configurations.

Resolution	CUDA Grid Dimensions	CUDA block dimensions
256×256	8×8×1	8×8×1
512×512	16×16×1	16×16×1
1024×1024	32×32×1	32×32×1
2048×2048	64×64×1	32×32×1
4096×4096	128×128×1	32×32×1
8192×8192	256×256×1	32×32×1



❌ Performance Limitations

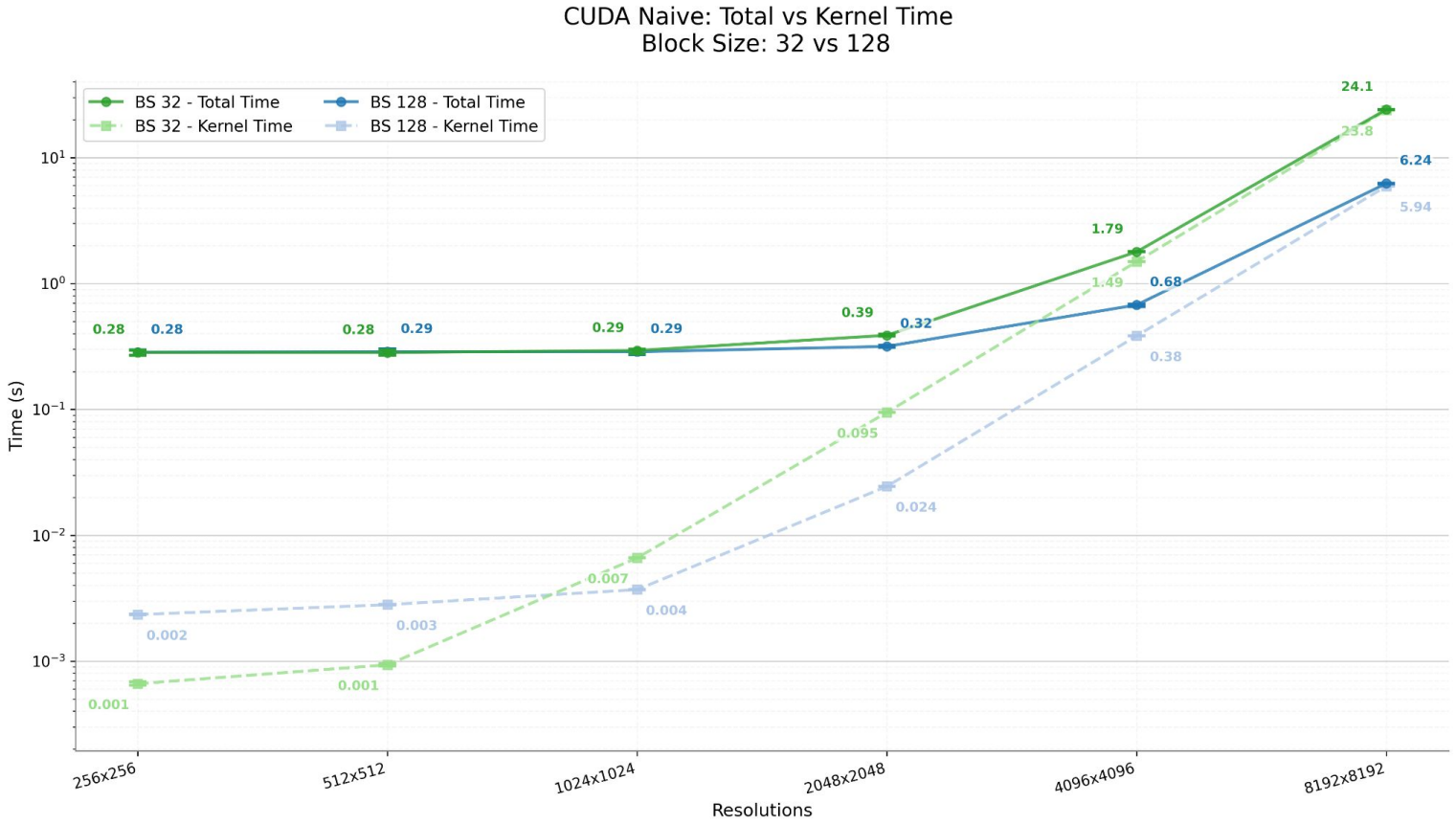
At standard resolutions, the excessive GMEM transfer overhead masks any computational gains: while the kernel itself consistently outperforms the CPU, the total execution time (including H2D/D2H transfers) can exceed CPU-based approaches at lower resolutions.

CUDA Naive: benchmark

Performance improvement: increasing the block size drastically shrinks the CUDA grid, minimizing the total number of blocks launched. This quadratically reduces both the number of GMEM write operations and the size of the resulting motion vector array.

Sequential Thread Overload: as threads parallelize the search across candidate positions, increasing the block size forces each thread to sequentially process a quadratically larger number of pixels.

Block size (resolution 2048×2048))	CUDA Grid Dimensions	CUDA block dimensions
32	64×64×1	32×32×1
128	16×16×1	16×16×1



⚠️ **Accuracy vs. Performance Trade-off**

While increasing the block size improves execution time, it severely degrades the granularity of the motion vectors, leading to a highly approximated result.

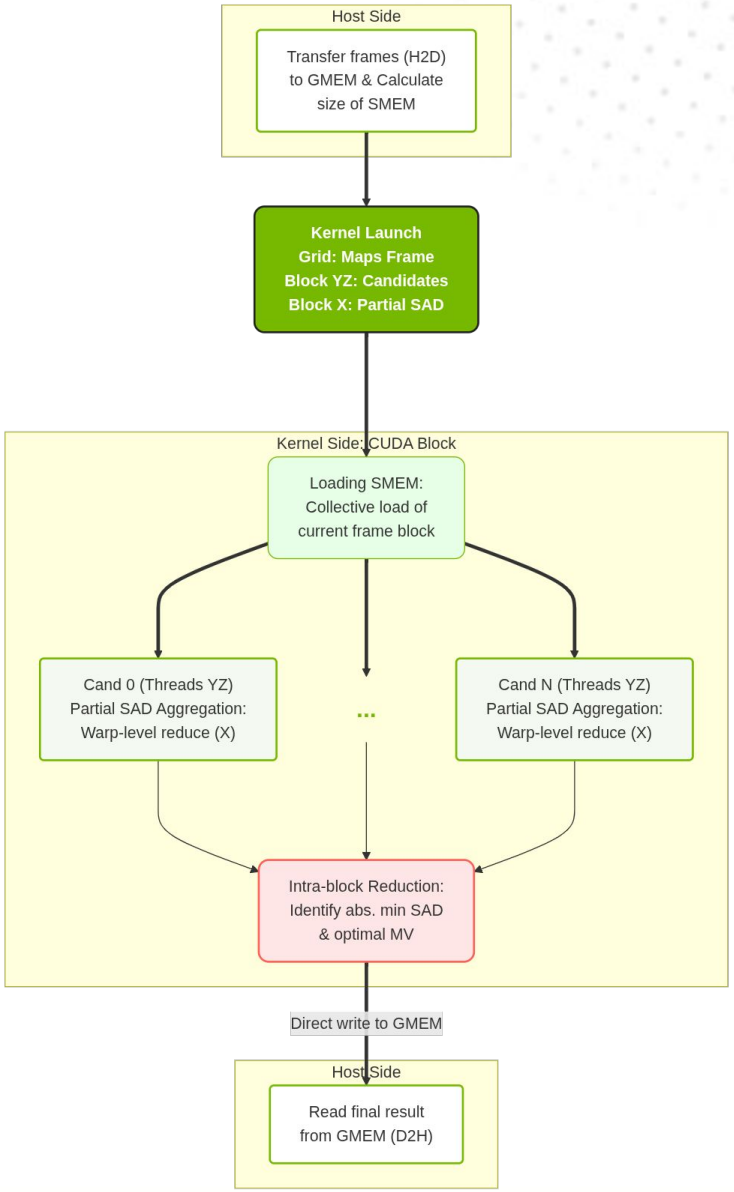
CUDA Optimized Uncoalesced

To improve block-by-block parallelization, this implementation introduces granular parallelism for SAD computation inside di cuda block:

- **CUDA grid 2D:** maps the frame grid. Each CUDA Block processes exactly one block of the current frame.
- **CUDA block 3D:** YZ axes map to the search window candidate positions, while threads along the X-axis concurrently compute the partial SAD for the same pair of frame blocks.

Workflow idea:

1. **Host Side**: transfer (H2D) the current and reference frames into GMEM and calculate size of the SMEM.
2. **Kernel Side:**
   a. **Loading SMEM**: each thread collectively loads the current frame block associated to its respective CUDA block into SMEM.
   b. **Intra-Candidate Reduction:** compute partial SAD and, by cooperative-groups reduction, accumulate values into a total SAD for each candidate reference frame block.
   c. **Block-Wide Reduction**: cooperative reduction across warps to identify the candidate with minimum SAD and optimal MV, followed by a direct write of the final result to GMEM.
3. **Host Side:** read final result from GMEM (D2H).



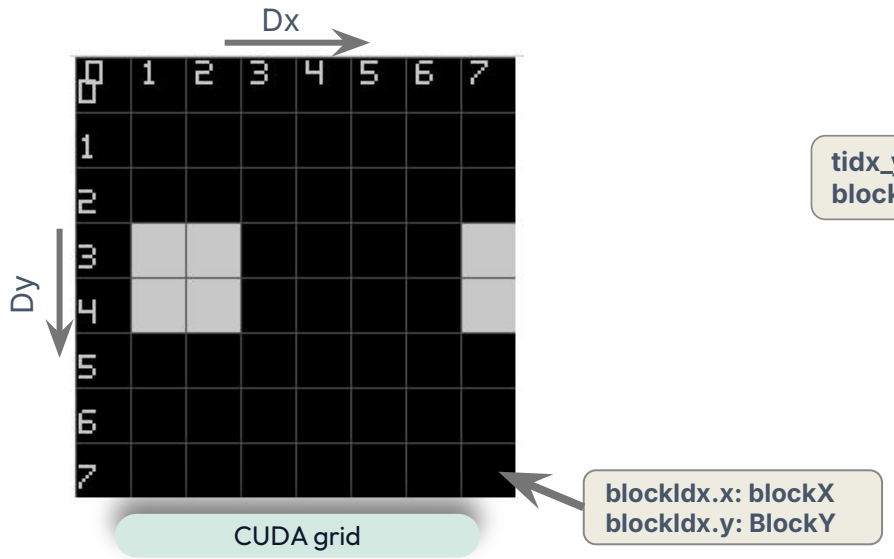
CUDA Optimized Uncoalesced: grid and block dimensions

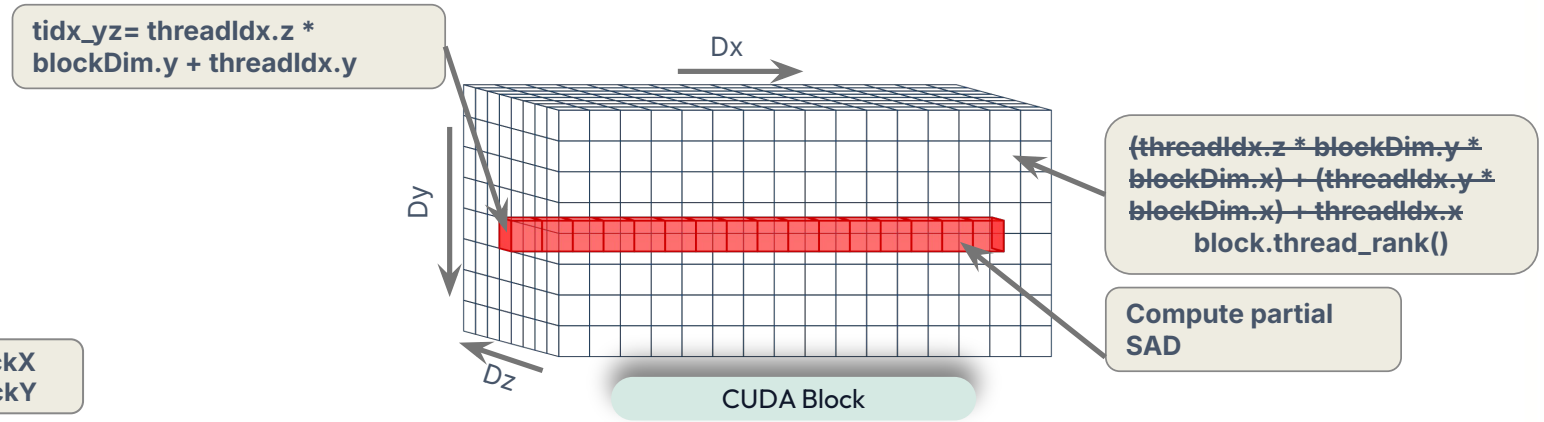
GridDim: 2 dimensions [blocksX, blocksY]

- The CUDA grid has the same dimensions as the frame's block grid.
- Each CUDA block is responsible for finding the best match for its corresponding current frame block within the reference frame.

BlockDim: 3 dimensions [maxThreadsPerBlock/(blocksX*blocksY), blocksX, blocksY]:

- Y and Z dimensions map candidate reference frame blocks.
- X dimension parallelizes SAD computation for each candidate pair, sized to the largest power of two within the remaining thread budget, capped at 32 to keep the intra-candidate reduction within a single warp.
- A fallback configuration (4 × 16 × 16) applies when YZ thread count exceeds hardware limits.





CUDA Optimized Uncoalesced: kernel (Intra-Candidate Reduction)

Each thread computes partial SAD of a pixel subset within its assigned candidate block:

1. Threads YZ iterate over all assigned candidate reference frame blocks. For each iteration, threads along X compute partial SAD between a pixel subset of the current frame block and the candidate reference frame block for that iteration.
2. Intra-Candidate Reduction: partial SAD values are aggregated into the total SAD for each candidate position via a cooperative-groups tile reduction.
3. Best Match Selection: each thread YZ with index 0 calculate the motion vector and update the best result among the assigned reference frame blocks.

fullSearchBM_optimized_uncoalesced.cpp: fullSearchCUDAUncoalescedOptimizedGray

```
// Precalculate pixels per thread for SAD parallelization along X dimension
const int pixelsPerThread = (blockSize * blockSize + blockDim.x - 1) / blockDim.x;

// Each YZ thread processes assigned candidate positions with stride = threadsYZ
// (threadsYZ = number of threads on YZ CUDA block dimensions)
for (int pos = tidx_yz; pos < blockX * blockY; pos += threadsYZ) {
 // Convert candidate position to reference frame block coordinates
 int ref_bx = pos % blocksX; int ref_by = pos / blocksX;
 // if current block or reference block pixels exceed frame dimension mark this
  candidate as invalid, otherwise, compute SAD only for pixels within frame bounds
 int partial_sad = computeSAD_device_partial(s_curr, d_ref,
  ref_bx * blockSize, ref_by * blockSize, blockSize, width,
   threadIdx.x * pixelsPerThread, // starting pixel index
   pixelsPerThread // pixels handled by thread
  );
 // Reduce partial SAD across X threads to get total SAD for current reference block
 int total_sad = reduce_partial_sad_warp(warp, partial_sad, blockDim.x);
 // Thread 0 of YZ updates best match if this candidate has lower SAD (or equal with
  lower distance)
}
// Threads 0YZ write local best match into shared memory (best_thread_results)
```

fullSearchBM_optimized_uncoalesced.cpp

```
__device__ __inline__ int computeSAD_device_partial(const unsigned char* curr,
 const unsigned char* ref, int ref_x, int ref_y, int blockSize, int width,
 int startPixel, int pixelsPerThread) {
  int sad = 0;// Compute SAD for pixels assigned to this thread
  for (int pixelIdx = startPixel; pixelIdx < startPixel + pixelsPerThread; pixelIdx++) {
    // Check if pixel index exceeds block boundary
    // Convert linear pixel index to 2D coordinates within the block
     int ry = ref_y + pixelIdx / blockSize; int rx = ref_x + pixelIdx % blockSize;
     // Accumulate SAD
     sad = __usad(curr[pixelIdx], ref[ry * width + rx], sad);
   }
   return sad;
}
```

CUDA Optimized Uncoalesced: kernel (Block-Wide Reduction)

After each thread YZ with index 0 stores its best match, a cooperative block-wide reduction is performed to find the optimal reference frame block for the current frame block assigned to this CUDA block:

1. Warp-Level Reduction: each warp processes a portion of the per-thread best results from SMEM, comparing candidates via warp-level reduction to find the best match per warp.
2. Warp Leader Storage: thread rank 0 of each warp writes its warp's best result to SMEM.
3. Final Block Reduction: warp 0 performs a final warp-level reduction among all warp results to determine the best match for the entire CUDA block.
4. Result Writing: thread 0 writes the final motion vector to GMEM.

fullSearchBM_optimized_uncoalesced.cpp: fullSearchCUDAUncoalescedOptimizedGray

```
//reduction: each warp scans its portion of best_thread_results
const int warp_id = block.thread_rank() / 32;
const int num_warps = (blockDim.x * blockDim.y * blockDim.z + 31) / 32;

CandidateSad warp_best = {INT_MAX, 0, 0};
for (int i = warp_id * 32 + warp.thread_rank(); i < threadsYZ; i += num_warps
* 32) {
    CandidateSad candidate = best_thread_results[i];
    // update warp_best if candidate is better
}
warp_best = reduce_best_candidate_warp(warp, warp_best);

// Warp leader writes result to shared memory
if (warp.thread_rank() == 0)
    warp_results[warp_id] = warp_best;
block.sync();

// Final reduction: warp 0 finds best among all warp leaders
CandidateSad block_best = {INT_MAX, 0, 0};
if (warp_id == 0) {
    if (warp.thread_rank() < num_warps)
        block_best = warp_results[warp.thread_rank()];
    block_best = reduce_best_candidate_warp(warp, block_best);
}

// Thread 0 writes final motion vector to GMEM
```

CUDA Optimized Uncoalesced: profiling

Profiling Sad computation with frame of 2048×2048 and block size of 32 (1024 pixels), so frame grid has 64×64 frame blocks:

- **Frame size:** 4096 KiB
- **CUDA grid**: 64×64×1 blocks.
- **CUDA block:** 4×16×16 threads.
- **SMEM size:** 4.375 KiB.

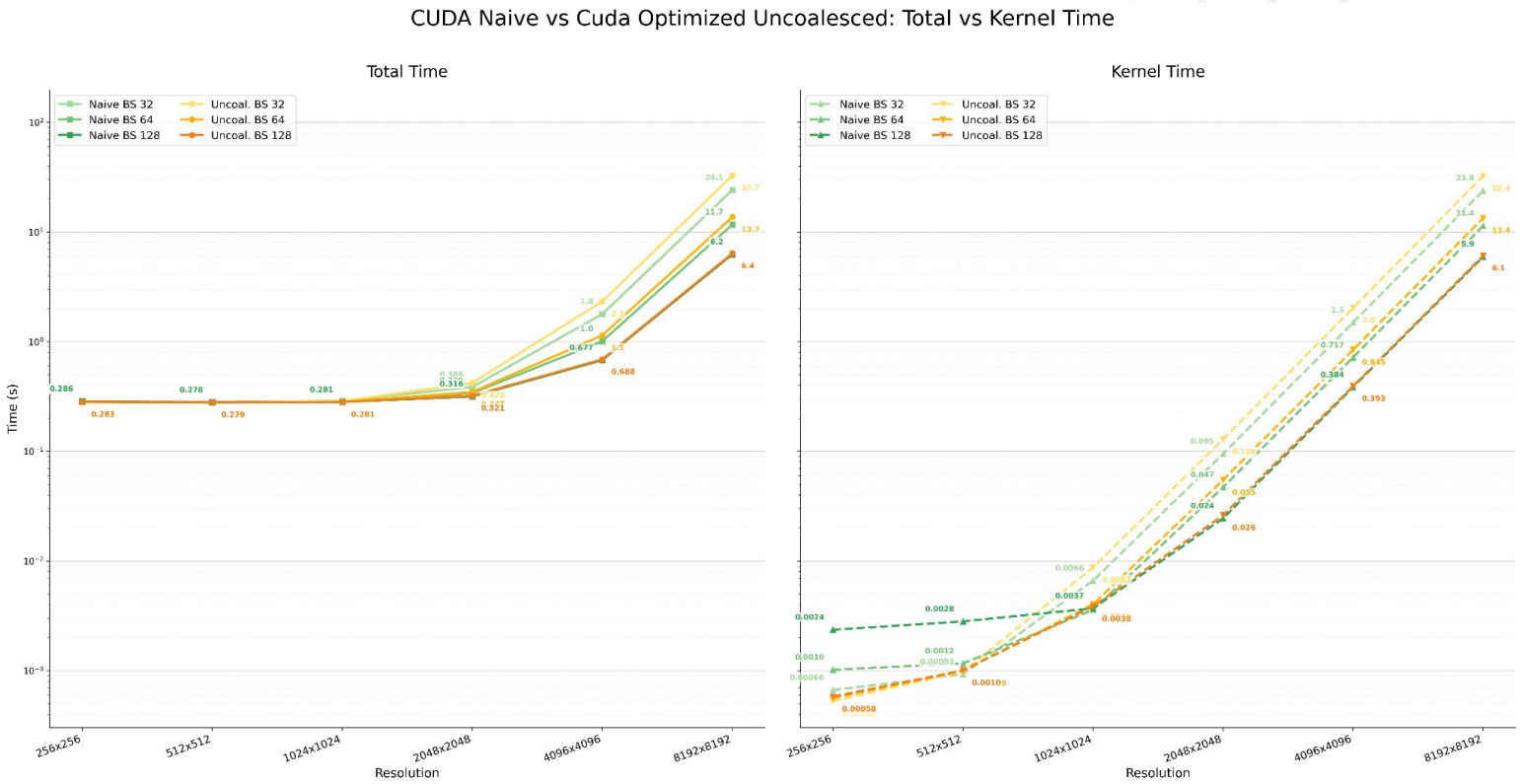
Profiling

- Compute (SM) throughput: 61.69% & Memory Throughput: 96.55%: memory-bound.
- L1 Cache Throughput: 97.79% & L1 Cache Hit Rate: 96.87%:
  - L1 cache completely saturated despite high hit rate
- L2 Cache Throughput: 4.77% & L2 Cache Hit Rate: 99.92%:
  - L1 acts as primary bottleneck, leaving L2 largely idle.
- uncoalesced Accesses, 97% waste:
  - Non-coalesced pattern generates wasted memory sectors.
- Achieved Occupancy 49.99%: capped by register pressure and size SMEM.

CUDA Optimized Uncoalesced: benchmark

No performance improvement: while parallelizing the SAD computation along the Z-axis significantly accelerates the calculation of partial SADs, the subsequent reduction step required to obtain the final SAD introduces synchronization overhead that can limit overall performance improvements.

block size (Resolution 8192×8192)	CUDA Grid Dimension	CUDA block dimensions	SMEM Size
32	256×256×1	4×16×16	8 KiB
64	128×128×1	4×16×16	11 KiB
128	64×64×1	4×16×16	23 KiB
256	32×32×1	1×32×32	80 KiB ❌
512	16×16×1	4×16×16	263 KiB ❌



❌ **Cannot increase block size indefinitely**

Since every frame block is loaded into SMEM, increasing the block size drastically increases the bytes occupied per block, quickly exceeding physical CUDA limits.

CUDA Optimized

To improve block-by-block parallelization and coalescing access to GMEM, is necessary that each cuda block access also to only one block reference frame:

- **CUDA grid 3D:** the first 2 axes XY maps the frame grid. Simultaneously, the Z axis maps to the candidate blocks in the reference frame.
- **CUDA block 1D**: parallelize the SAD computation.

❌ **Reduction problem**

Since SAD values are spread across the Z axis, finding the best SAD and motion vector requires a reduction step. Reusing the same 3D grid for this would lead to low SM occupancy.

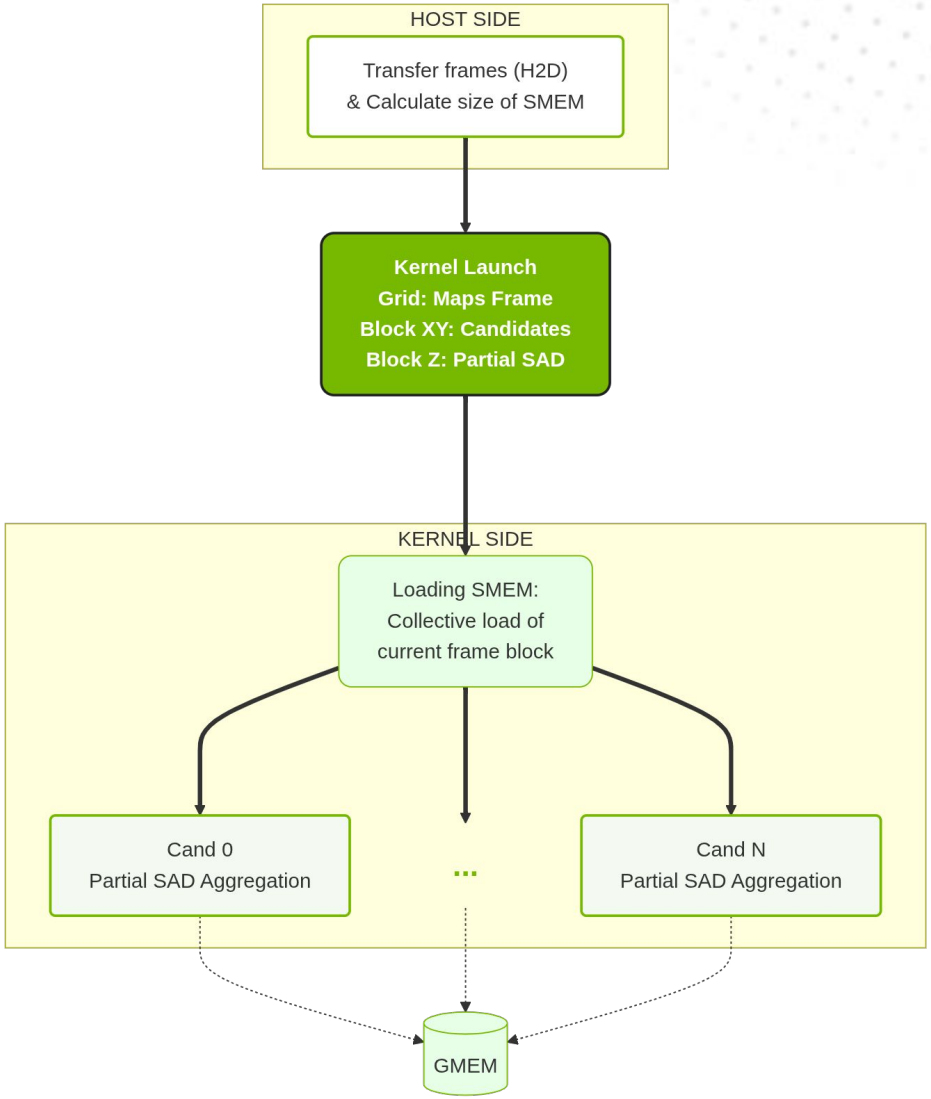
Workflow idea:

1. **Host Side**:
   a. Transfer (H2D) the current and reference frames into GMEM.
   b. Dynamically evaluate available SMEM to select the optimal caching strategy.
2. **Kernel Side:**
   a. **Loading SMEM**: threads cooperatively load current and reference frame blocks into SMEM.
   b. **Intra-Candidate Reduction:** compute partial SAD and, via blockReduce, accumulate values into a total SAD for each candidate reference frame block.
   c. **Grid-Wide Reduction**: cooperative grid reduction ~~of the candidates with minimum optimized by a direct write of the final result to GMEM.~~
3. **Host Side:** read final result from GMEM (D2H).

CUDA Optimized: computeSADKernel

Workflow idea:

1. **Host Side**:
   a. Transfer (H2D) the current and reference frames into GMEM.
   b. Dynamically evaluate available SMEM to select the optimal caching strategy.
2. **Kernel Side:**
   a. **Loading SMEM**: threads cooperatively load current and reference frame blocks into SMEM.
   b. **Intra-Candidate Reduction:** compute partial SAD and, via BlockReduce, accumulate values into a total SAD for each candidate reference frame block. Each SAD result is written to GMEM.



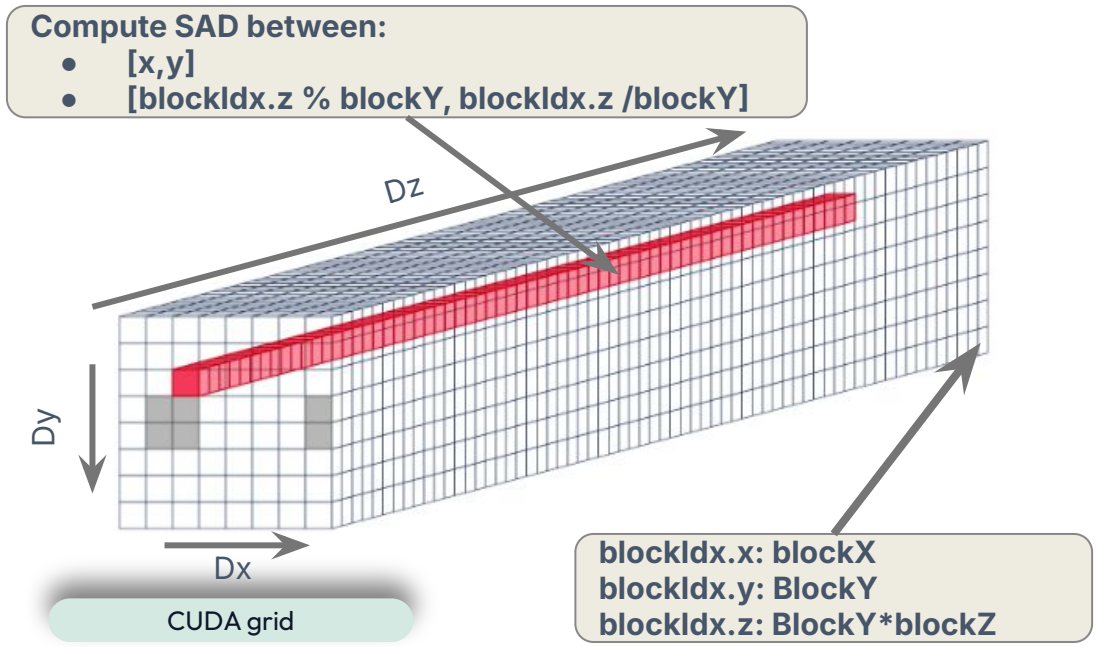
CUDA Optimized: computeSADKernel grid and block dimensions

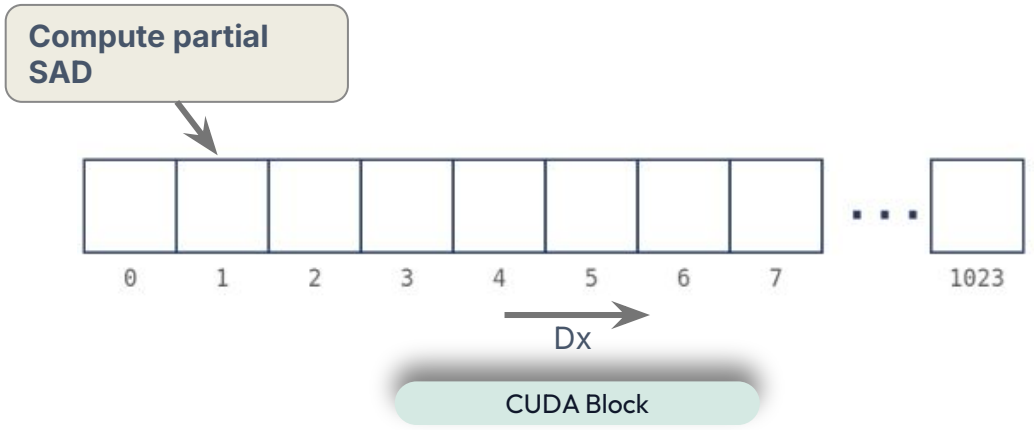
GridDim: 3 dimensions [blockX, blockY, blockX*BlockY]

- XY Dimensions map directly to the position of the block within the current frame.
- Z Dimension parallelizes the search workload. Each block computes the SAD value between the current XY block and its assigned reference frame block.

BlockDim: 1 dimension [block size*block size]

- The pixels of the current and reference frame block pair assigned to this CUDA block are divided among threads, each thread calculates the partial SAD for its assigned subset of pixels.
- The host maximizes the thread count up to the hardware limit or total pixel count, strictly enforcing a power of 2 for a safe parallel reduction.





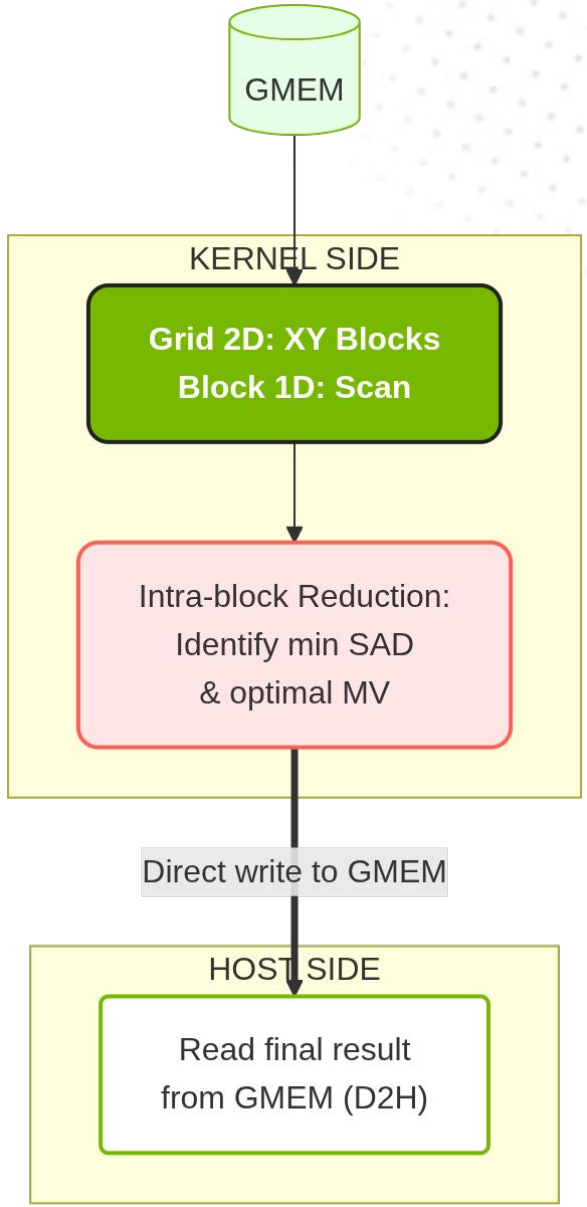
CUDA Optimized: findBestMVKernel

Second kernel executes a reduction on the results generated by the first kernel:

- **CUDA grid 2D:** maps the frame grid. Each CUDA block processes exactly one block of the current frame, comparing SAD results.
- **CUDA block 1D:** each thread processes an assigned subset of candidate SADs.

Workflow idea:

1. **Kernel Side:**
   a. **Thread local reduction**: each thread independently scans its assigned portion of the previously computed results, isolating its local best candidate.
   b. **Block-Wide Reduction:** cooperative reduction to identify the candidate with minimum SAD and optimal MV, followed by a direct write of the final result to GMEM.
2. **Host Side:** read final result from GMEM (D2H).



CUDA Optimized: computeSADKernel host

GMEM structures:

- d_sad: intermediate array, fully populated by the end of the first kernel, storing the computed SAD values for every current frame block across all its search window.

fullSearchBM_optimized.cpp: fullSearchCUDAOptimizedGray

```
// Allocate and copy d_curr and d_ref
// Allocate device memory for SAD values and motion vectors
const size_t sadBytes = (size_t)(blocksX * blocksY)^2 *
 sizeof(int);
int* d_sad = nullptr;
gpuErrorCheck(cudaMalloc(&d_sad, sadBytes));
```

SMEM layout: available SMEM is evaluated dynamically after reserving the static overhead required by the CUB library. The kernel adopts one of three strategies:

- FULL: curr + ref frame blocks (*smTwoFrames*) fit in SMEM.
- CURR: only curr frame block (smOneFrame) fit in SMEM.
- NONE: neither cur and ref frame fit in SMEM.

fullSearchBM_optimized.cpp: launchSADKernel

```
constexpr size_t cubSmem  = sizeof(cub::BlockReduce<
 int, KSAD_THREADS>::TempStorage);
const size_t KSAD_smem_available =
 deviceProp.sharedMemPerBlock - cubSmem;
const size_t smOneFrame   = pixelsPerBlock * sizeof(unsigned char);
const size_t smTwoFrames  = 2 * smOneFrame;

if (smTwoFrames <= KSAD_smem_available) {
 //FULL: enough SMEM for both current and reference blocks
 strategy = SmemStrategy::FULL; smKSad = smTwoFrames;
 // launch kernel (switch threadsKSad)
}
else if (smOneFrame <= KSAD_smem_available) {
 //CURR_ONLY: enough SMEM for current block only
 strategy = SmemStrategy::CURR_ONLY; smKSad = smOneFrame;
 // launch kernel (switch threadsKSad)
}
else {
 // not enough SMEM for frame blocks,
 strategy = SmemStrategy::NONE; smKSad = 0;
 // launch kernel (switch threadsKSad)
}
```

CUDA Optimized: computeSADKernel device

Each thread computes partial SAD of a pixel subset within its assigned candidate reference block:

1. Threads iterate over pixels of the assigned candidate block, using the selected strategy to read current and reference pixels:
   a. FULL: both blocks are cooperatively loaded into SMEM beforehand; pixels are read from SMEM.
   b. CURR_ONLY: the current block is read from SMEM; the reference block is read from GMEM.
   c. NONE: both blocks are read directly from GMEM.
2. Intra-Candidate Reduction: partial SAD values are aggregated via cub::BlockReduce to compute the total SAD for this candidate position.
3. Thread 0 writes the total SAD to GMEM.

fullSearchBM_optimized.cpp: computeSADKernel

```
// FULL and CURR strategy: load current frame block into SMEM for FULL
if constexpr (STRATEGY != SmemStrategy::NONE) {
 for (int i = threadIdx.x; i < pixelsPerBlock; i += KSAD_THREADS)
  s_curr[i] = d_curr[(y + i / blockSize) * frame_pixel_width + (x + i % blockSize)];
    __syncthreads();
}

// Each Z block processes assigned candidates, stride = gridDim.z
for (int flat_idx = blockIdx.z; flat_idx < blockX * blockY; flat_idx += gridDim.z) {

 // flat index -> reference block (x, y), in pixels
 const int ref_x = (flat_idx % blockX) * blockSize;
 const int ref_y = (flat_idx / blockY) * blockSize;

 // FULL strategy: cooperatively load reference block into SMEM

 int partial_sad = 0;
 for (int i = threadIdx.x; i < pixelsPerBlock; i += KSAD_THREADS) {
  unsigned char curr_frame_pixels, ref_frame_pixels;
  if (STRATEGY == FULL)             // load curr + ref from SMEM
  else if (STRATEGY == CURR_ONLY)   // load curr from SMEM, ref from GMEM
  else                              // load curr + ref from GMEM

  partial_sad = __usad(curr_frame_pixels, ref_frame_pixels, partial_sad);
 }
 // reduce partial SADs across threads -> total SAD for this candidate
 const int total_sad = BlockReduce(partial_sad_reduce_storage).Sum(partial_sad);
  if (threadIdx.x == 0)
   d_sad[(blockIdx.y * gridDim.x + blockIdx.x) * (blockX * blockY) + flat_idx] =
total_sad;
  __syncthreads();
}
```

CUDA Optimized: computeSADKernel profiling

Profiling Sad computation with 1024×1024 and block size of 32 (1024 pixels), so frame grid has 32×32 frame blocks

- **Frame size:** 1024 KiB
- **CUDA grid**: 32×32×1024 blocks.
- **CUDA block:** 1024×1×1 threads.
- **Memory:** SMEM 2 KiB, *d_sad* (GMEM) 4 MiB

Profiling

- Compute (SM) throughput: 54.35% & Memory Throughput: 11.38%:
  - Moderate compute utilization with minimal memory pressure.
- L1 Cache Throughput: 12.21% & L1 Cache Hit Rate: 47.98%:
  - The 52.02% miss rate reflects reference block diversity, each candidates in the Z dimension reads different reference regions.
- L2 Cache Throughput: 1.36% & L2 Cache Hit Rate: 99.76%:
  - Adjacent candidates in the Z dimension access andiacent reference frame block.
- GMEM Throughput: 0.01% & Achieved Occupancy: 49.18%
  - Occupancy of 49.18% is limited by barrier synchronization overhead during BlockReduce stages.

CUDA Optimized: findBestMVKernel device

Each thread finds the local best SAD among its assigned candidates:

1. Threads iterate over their assigned candidates, reading each SAD value from GMEM and keeping the best candidate found so far, with tie-breaking on motion vector distance.
2. Block-Wide Reduction: local best candidates are aggregated via cub::BlockReduce to find the overall best candidate for this block.
3. Thread 0 converts the winning candidate into a motion vector and writes it to GMEM.

fullSearchBM_optimized.cpp: findBestMVKernel

```
// base offset into d_sad for this block's candidates
const size_t base = (blockIdx.y * gridDim.x + blockIdx.x) * (blockX*blockY);

CandidateSadIdx local_best = {INT_MAX, -1};
const CandidateSadOp op(blockIdx.x, blockIdx.y);  // full search: no window offset needed

// each thread scans a subset of candidates, stride = BLOCK_REDUCE_THREADS
for (int flat_idx = threadIdx.x; flat_idx < blockX * blockY; flat_idx +=
BLOCK_REDUCE_THREADS) {
 const CandidateSadIdx candidate = {
  d_sad[base + flat_idx],  // SAD value for this candidate, from GMEM
  flat_idx                 // candidate index, used to recover its position later
 };
 local_best = op(local_best, candidate);  // keep the better of local_best and candidate
}

// Block-Wide Reduction: find the best candidate among all threads
using BlockReduce = cub::BlockReduce<CandidateSadIdx, BLOCK_REDUCE_THREADS>;
__shared__ typename BlockReduce::TempStorage reduce_storage;
const CandidateSadIdx result = BlockReduce(reduce_storage).Reduce(local_best, op);

if (threadIdx.x == 0) {
 // convert winning candidate index back to reference frame block coordinates
 const int ref_x = result.flat_idx % blockX;
 const int ref_y = result.flat_idx / blockY;
 // motion vector = displacement between reference and current block
 d_mv[blockIdx.y * gridDim.x + blockIdx.x] = { ref_x - blockIdx.x, ref_y - blockIdx.y};
}
```

CUDA Optimized: findBestMVKernel profiling

Profiling Sad computation with 1024×1024 and block size of 32 (1024 pixels), so frame grid has 32×32 frame blocks

- **Frame size:** 1024 KiB
- **CUDA grid**: 32×32×1 blocks.
- **CUDA block:** 1024×1×1 threads.

Profiling

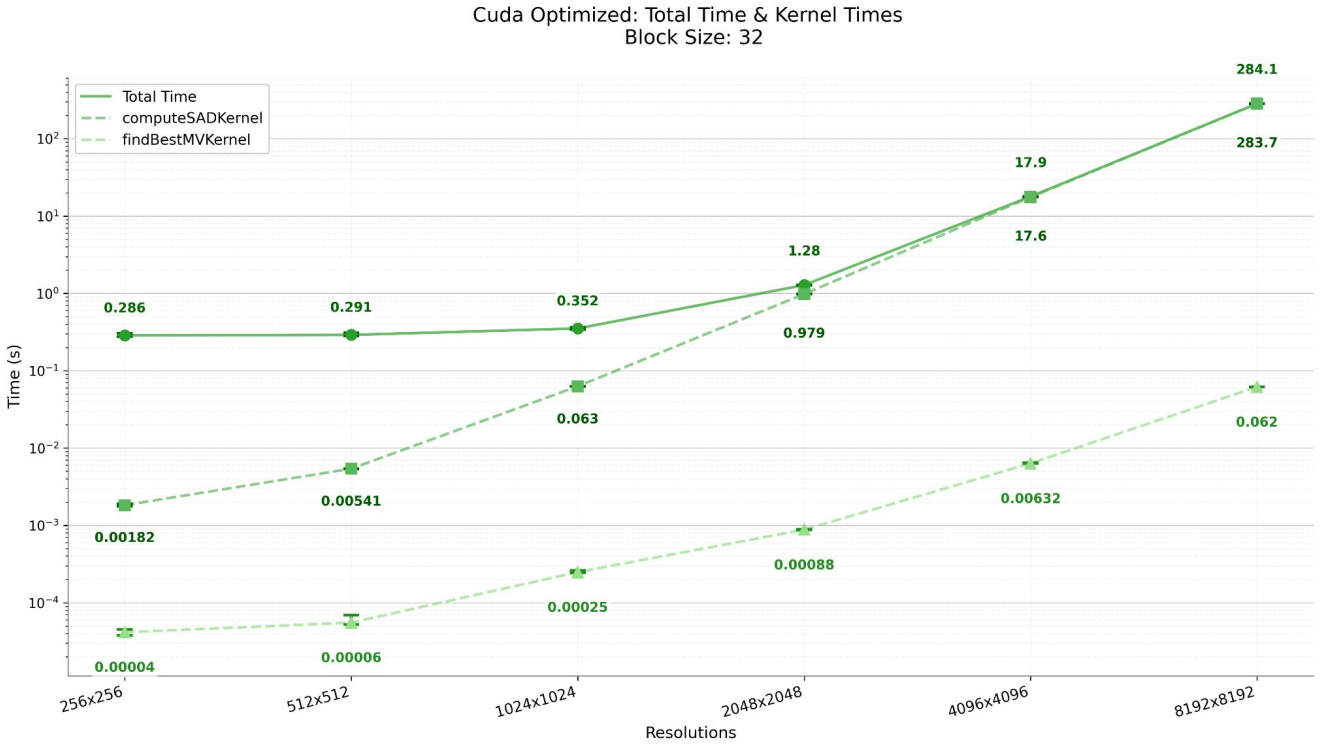
- Compute (SM) throughput: 44.71% & Memory Throughput: 5.66%:
  - Low utilization, performs only minimal arithmetic (comparisons, indexing) with high memory access frequency.
- L1 Cache Throughput: 6.02% & L1 Cache Hit Rate: 0.81%:
  - Each of 1024 threads reads different SAD values from d_sad array sequentially.
- L2 Cache Throughput: 2.28% & L2 Cache Hit Rate: 4.14%:
  - L2 cannot buffer the random-access pattern of SAD reads.
- GMEM Throughput: 2.64% & Achieved Occupancy: 39.82%

CUDA Optimized: benchmark

computeSADKernel (Compute-Bound): as resolution increases and number of frame blocks increase, mathematical operations dominate the execution time.

findBestMVKernel (Memory-Bound): a lightweight reduction step. Performance is strictly dictated by GMEM access latency.

Resolution (block size 32)	CUDA grid and block *ComputeSADKernel*	CUDA grid and block FindBestMVKernel	GMEM
256×256	grid: 8 × 8 × 64 block: 1024 × 1 × 1	grid: 8 × 8 × 1 block: 64 × 1 × 1	d_sad: 16 KiB d_mv: 0.5 KiB
512 × 512	grid: 16 × 16 × 256 block: 1024 × 1 × 1	grid: 16 × 16 × 1 block: 256 × 1 × 1	d_sad: 256 KiB d_mv: 2 KiB
1024 × 1024	grid: 32 × 32 × 1024 block: 1024 × 1 × 1	grid: 32 × 32 × 1 block: 1024 × 1 × 1	d_sad: 4 MiB d_mv: 8 KiB
2048 × 2048	grid: 64 × 64 × 4096 block: 1024 × 1 × 1	grid: 64 × 64 × 1 block: 1024 × 1 × 1	d_sad: 64 MiB d_mv: 32 KiB
4096 × 4096	grid: 128 × 128 × 16384 block: 1024 × 1 × 1	grid: 128 × 128 × 1 block: 1024 × 1 × 1	d_sad: 1024 MiB d_mv: 128 KiB
8192 × 8192	grid: 256 × 256 × 65535 block: 1024 × 1 × 1	grid: 256 × 256 × 1 block: 1024 × 1 × 1	d_sad: 16384 MiB d_mv: 512 KiB



SMEM usage

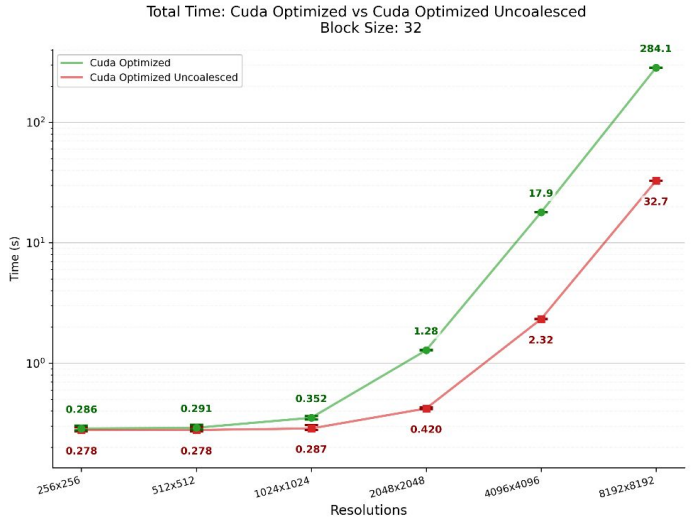
Shared Memory footprint is strictly proportional to the frame block size and after the selected search strategy.

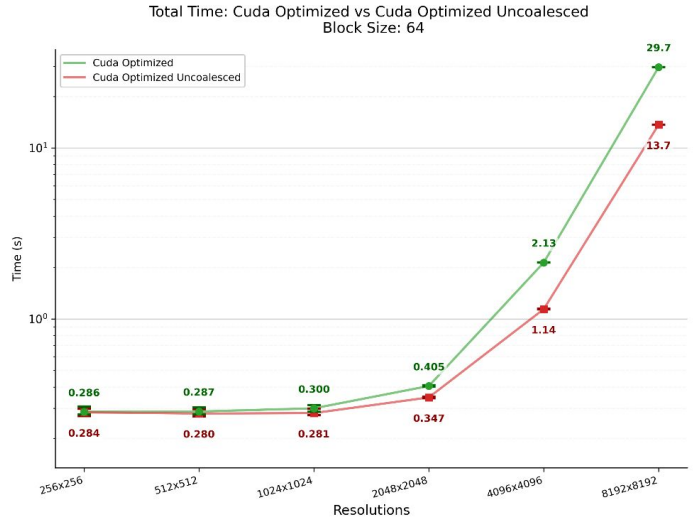
CUDA Optimized: benchmark

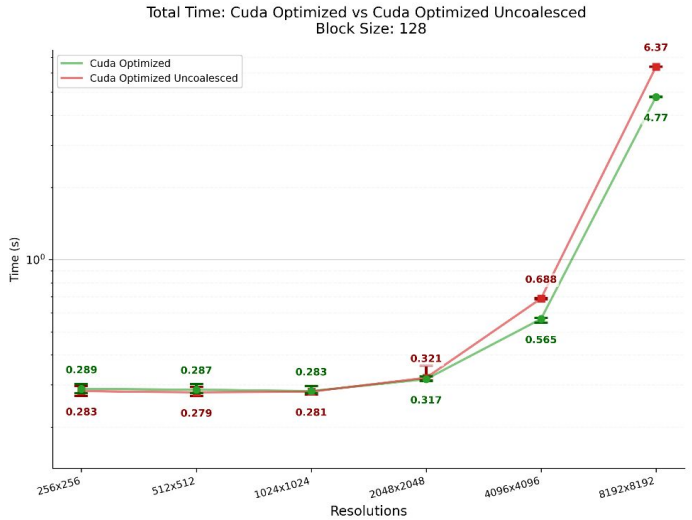
SMEM limits: full strategy support up to block size 128 (32 KiB). At block size 256, the hardware limit is exceeded.

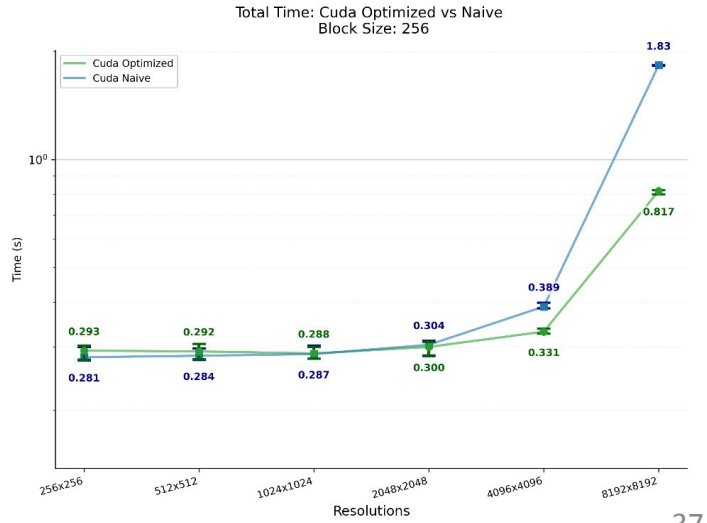
GMEM: the *d_sad* array dictates performance. Decreasing the block size triggers a quadratic growth in memory allocation.

block size (resolution 8192×8192)	SMEM for frame blocks (computeSADKernel)	d_sad (GMEM) size
32	FULL: 2 KiB	d_sad: 16384 MiB d_mv: 512 KiB
64	FULL: 8 KiB	d_sad: 1024 MiB d_mv: 128 KiB
128	FULL: 32 KiB	d_sad: 64 MiB d_mv: 32 KiB
256	NONE: 0 KiB	d_sad: 4 MiB d_mv: 8 KiB









Range Search

Comparing range search with full Search and OpenMP optimization

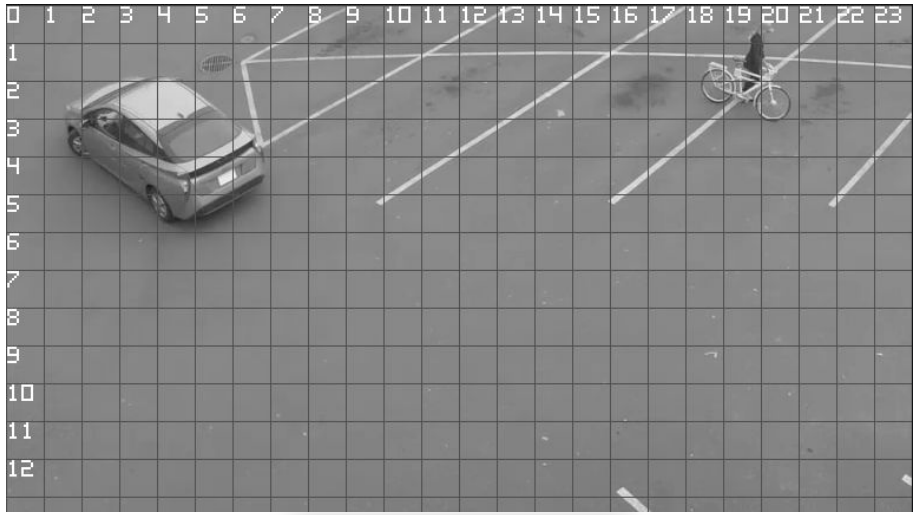
Full Search vs Range Search

Full Search:

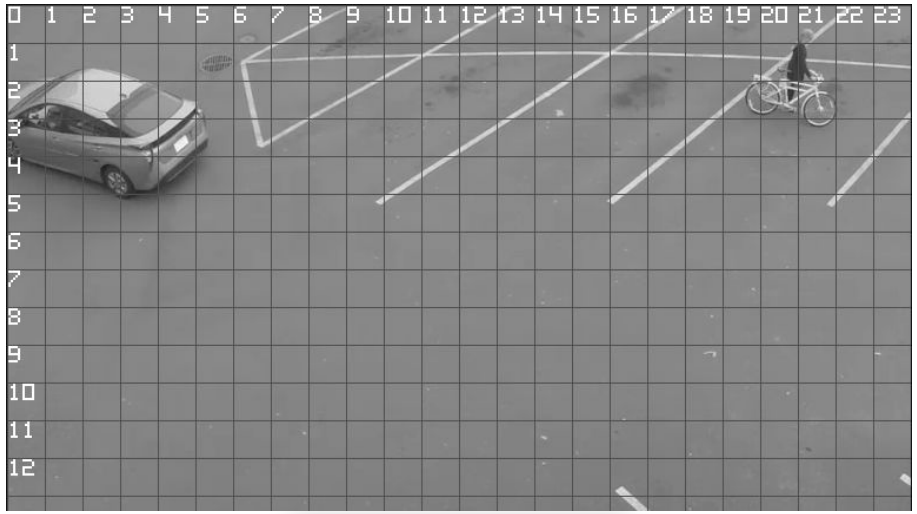
- Evaluates every possible block in the entire frame. Guarantees finding the mathematical global minimum SAD.
- Highly computational computational cost to handle frame with higher resolutions.

Range Search:

- Localized approach restricting candidate blocks. Significantly improves performance by reducing search complexity.
- Does not guarantee finding the global minimum SAD across the entire frame, but gains:
  - Noise Reduction: ignores distance matches.
  - Spatial Constraint: acts as a regularizer, forcing motion vectors to remain physically plausible.



Reference Frame



Current frame

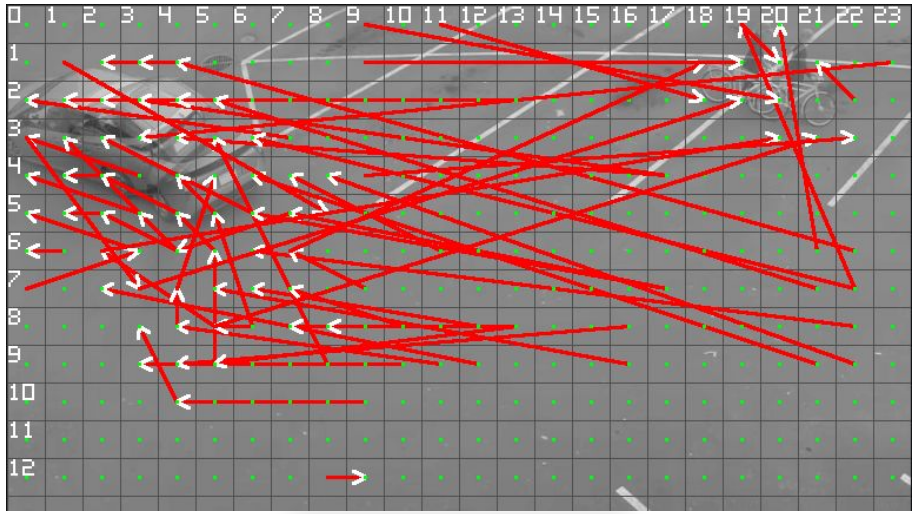
Full Search vs Range Search

Full Search:

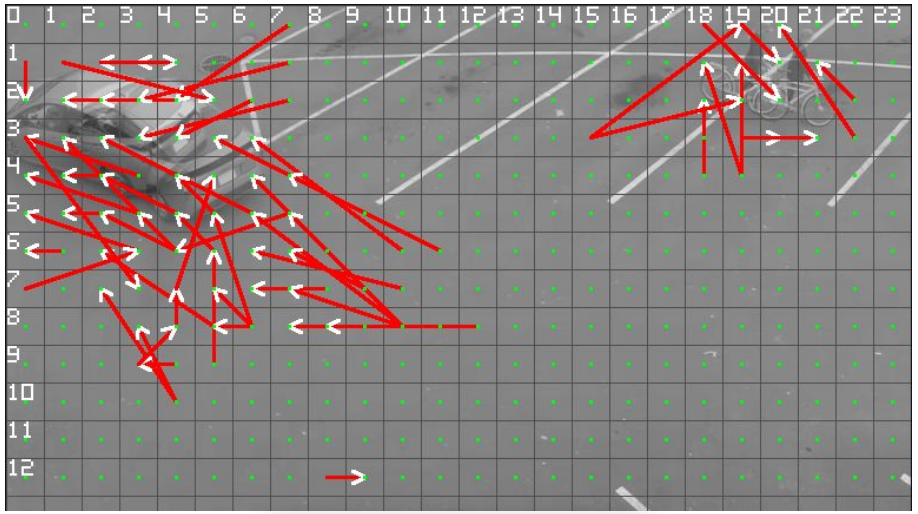
- Resolution: 768×432.
- block size: 32.
- Current frame blocks: 24 × 13.
- Window search area: 24 × 13 blocks.

Range Search:

- Resolution: 768×432
- block size: 32
- Current frame blocks: 24 × 13
- Range: 4
- Window search area: 9 × 9.



Full Search Motion Vector



Range search Motion Vector

CUDA kernel load imbalance

Each CUDA implementation maps the CUDA block dimensions to the maximum search window size.

A thread stays idle if its linear index (*tidx*) exceeds the number of valid candidate in the search window.

$$T_{idle} = max(0, threads_{block} - candidates_{window})$$

Central Block: $T_{idle} = 0$
The search window fits entirely within the frame, every thread runs at least one iteration, so hardware utilization is full.

Border Blocks: $T_{idle} > 0$
The search window gets clipped at the frame edges. Fewer candidate positions means the last threads in the block stay idle.

❌ **Warp Divergence**

At the borders, the shrinking search window leaves some threads idle before others, warp divergence only hits warps split between active and idle threads.

fullSearchBM_cuda_naive.cpp: fullSearchNaiveKernel

```
// linear thread id within the block
int tidx = threadIdx.y * blockDim.x + threadIdx.x;
int constexpr threadsPerBlock = threadsPerBlockDim * threadsPerBlockDim;
// current block top-left corner, in pixels
int blockX = blockIdx.x * blocksize;
int blockY = blockIdx.y * blocksize;
// search window for this block
CudaSearchBounds bounds = calculateCudaSearchBounds(blockIdx.x,
 blockIdx.y,gridDim.x, gridDim.y, searchRange);

// initialize best match found by this thread

for (int pos = tidx; pos < bounds.totalPositions; pos += threadsPerBlock) {
 ...
```

cuda_utils.h

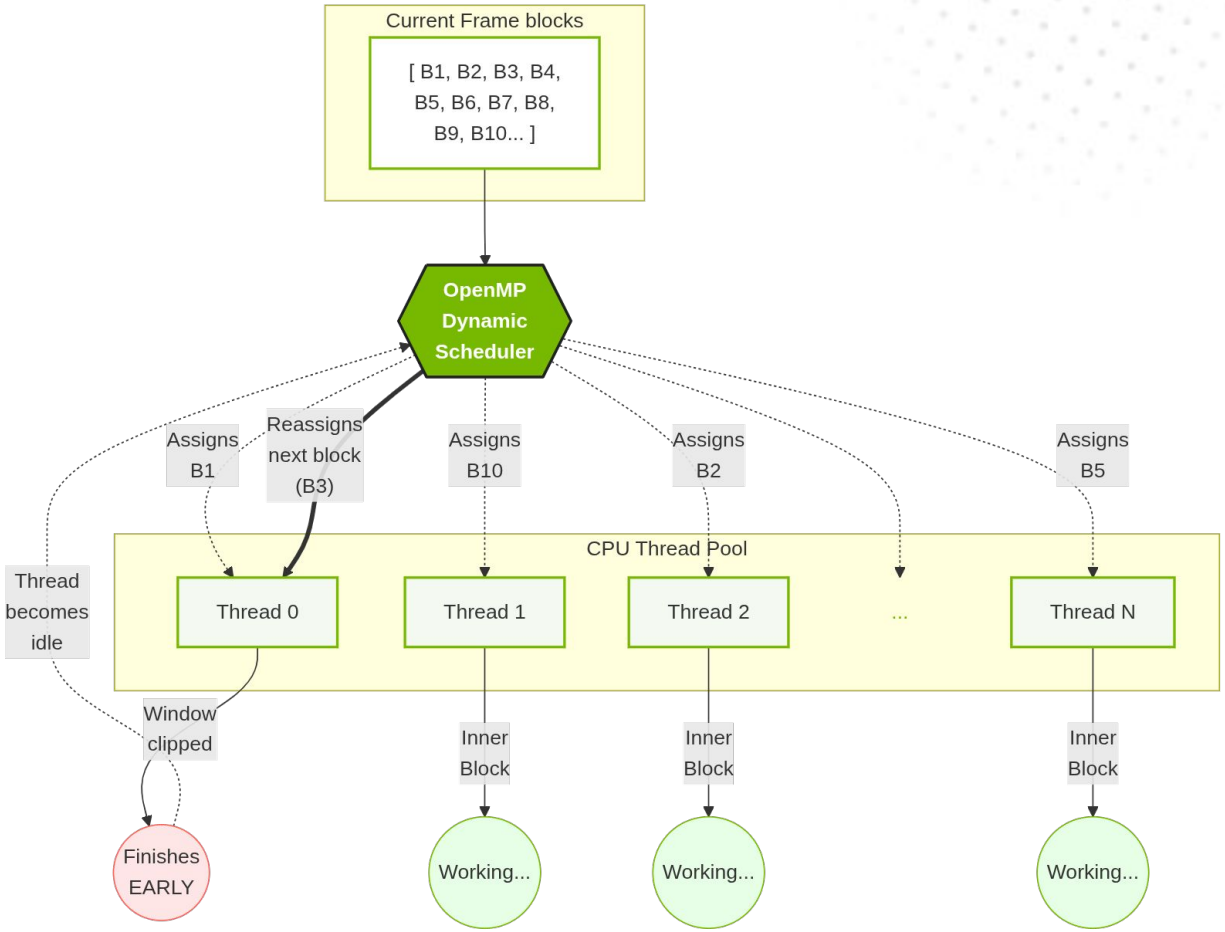
```
CudaSearchBounds calculateCudaSearchBounds(
 int blockIdxX, int blockIdxY, int gridDimX, int gridDimY, int searchRange) {

 CudaSearchBounds bounds;

 if (searchRange > 0) { \\range search
  bounds.startX = max(0, blockIdxX - searchRange);
  bounds.endX = min(gridDimX - 1, blockIdxX + searchRange);
  bounds.startY = max(0, blockIdxY - searchRange);
  bounds.endY = min(gridDimY - 1, blockIdxY + searchRange);
 } else { // full search }
  // number of candidate positions in the window
  bounds.width = bounds.endX - bounds.startX + 1;
  bounds.height = bounds.endY - bounds.startY + 1;
  bounds.totalPositions = bounds.width * bounds.height;
  return bounds;
}
```

CPU OpenMP

The OpenMP solution employs a Single Program, Multiple Data model (SPMD). A single block of code is executed in parallel by multiple threads, each thread operating on independent frame blocks.

Unlike CUDA's SIMT architecture, CPU cores are physically independent. This eliminates warp-level lock-step constraints.

In Range Search, blocks near the frame borders have fewer valid candidates than inner blocks, making the per-block workload non-uniform. The OpenMP runtime can dynamically reassign work to threads that finish early, rebalancing the load across cores.



CPU OpenMP: adaptive scheduling strategy

Profiling Factors:

- *borderRatio*: fraction of the frame affected by border clamping.
- *fragmentationFactor*: sets the optimal chunk size for the task queue.
- *costPerBlock*: cost of an inner block, in number of SAD operations.

Adaptive Scheduling Strategy: chooses static or dynamic scheduling based on cost per block and workload balance:

- **Static:** high cost per block, too few blocks to distribute, or uniform workload.
- **Dynamic:** non-uniform workload with low cost per block, balances border frame blocks.

fullSearchBM_cpu_OpenMP.cpp: fullSearchCPUOpenMPGray

```
int totalBlocks = blocksX * blocksY;
int numThreadsAvail = omp_get_max_threads();
int fragmentationFactor = 1;

if (searchRange > 0) {
 int innerBlocks = // blocks with a full search window
 float borderRatio = 1.0f - (float) innerBlocks / totalBlocks;
 // 0% border -> factor 1, 100% border -> factor 8
 fragmentationFactor = 1 + (int) (borderRatio * 7.0f);
}

int chunk = max(1, totalBlocks / (numThreadsAvail * fragmentationFactor));

if (searchRange <= 0) {
 omp_set_schedule(omp_sched_static, chunk);
} else {
 int costPerBlock = /* candidates for a center block */ * blockSize * blockSize;

 if (costPerBlock >= 50000 || totalBlocks < numThreadsAvail *
fragmentationFactor) {
  omp_set_schedule(omp_sched_static, chunk);
 } else {
  omp_set_schedule(omp_sched_dynamic, chunk);
 }
}
```

CPU OpenMP: two-level parallelism

Current-Frame Block-Level Thread Parallelism

collapse(2) flattens the two nested loops into a single iteration space, giving OpenMP more, smaller chunks of work to spread across threads, resulting in better load balancing, especially for the border blocks.

fullSearchBM_cpu_OpenMP.cpp: fullSearchCPUOpenMPGray

```
// collapse(2): merge loops
#pragma omp parallel for collapse(2) schedule(runtime)
for (int bx = 0; bx < blocksX; bx++) {
 for (int by = 0; by < blocksY; by++) {
  // best match so far
  MotionVector bestMV{0, 0};
  int bestSAD = INT_MAX, bestDist = INT_MAX;
  // candidate search range
  SearchBounds bounds = calculateSearchBounds(bx, by, blocksX, blocksY,
   searchRange);
  for (int irefY = bounds.startY; irefY <= bounds.endY; irefY++) {
   for (int irefX = bounds.startX; irefX <= bounds.endX; irefX++) {
    int dx = irefX - bx, dy = irefY - by;   // motion vector
    int sad = computeSAD(curr, ref, bx, by, irefX, irefY, blockSize);
    int dist = dx * dx + dy * dy;
    if (sad < bestSAD || (sad == bestSAD && dist < bestDist)) {
     bestSAD = sad; bestDist = dist; bestMV = {dx, dy};
    }
   }
  }
  mv[bx][by] = bestMV;
 }
}
```

Pixel-Level SIMD Parallelism

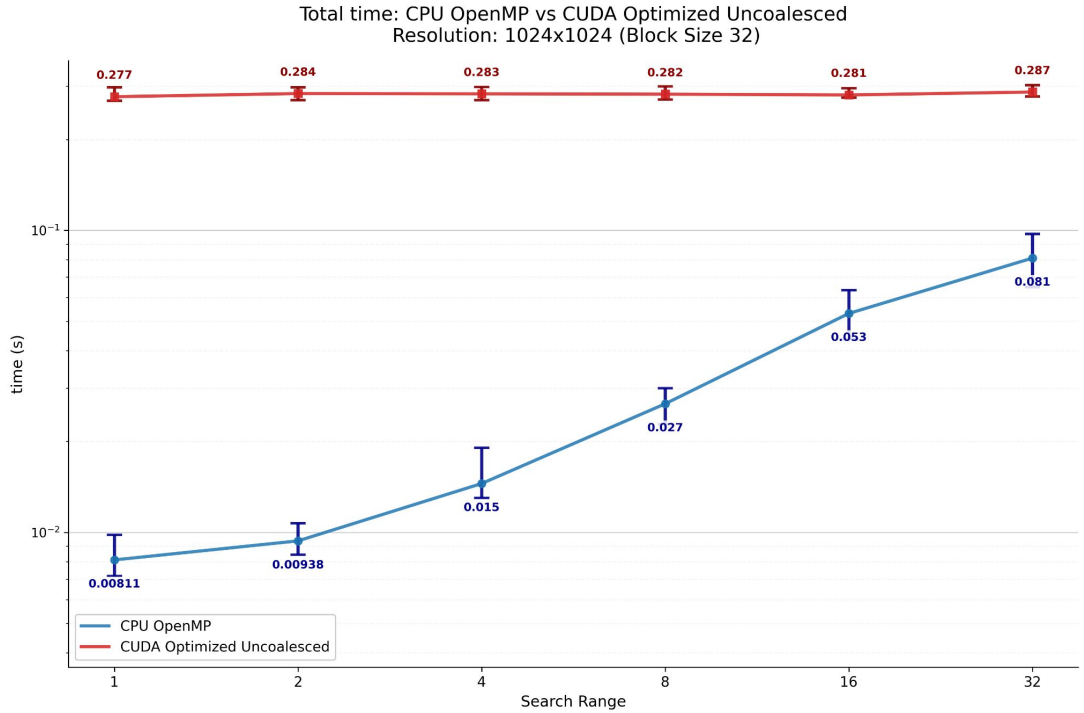
#pragma omp simd vectorizes the innermost SAD loop, telling the compiler to use CPU vector registers so it computes several pixel differences in a single instruction instead of one at a time.

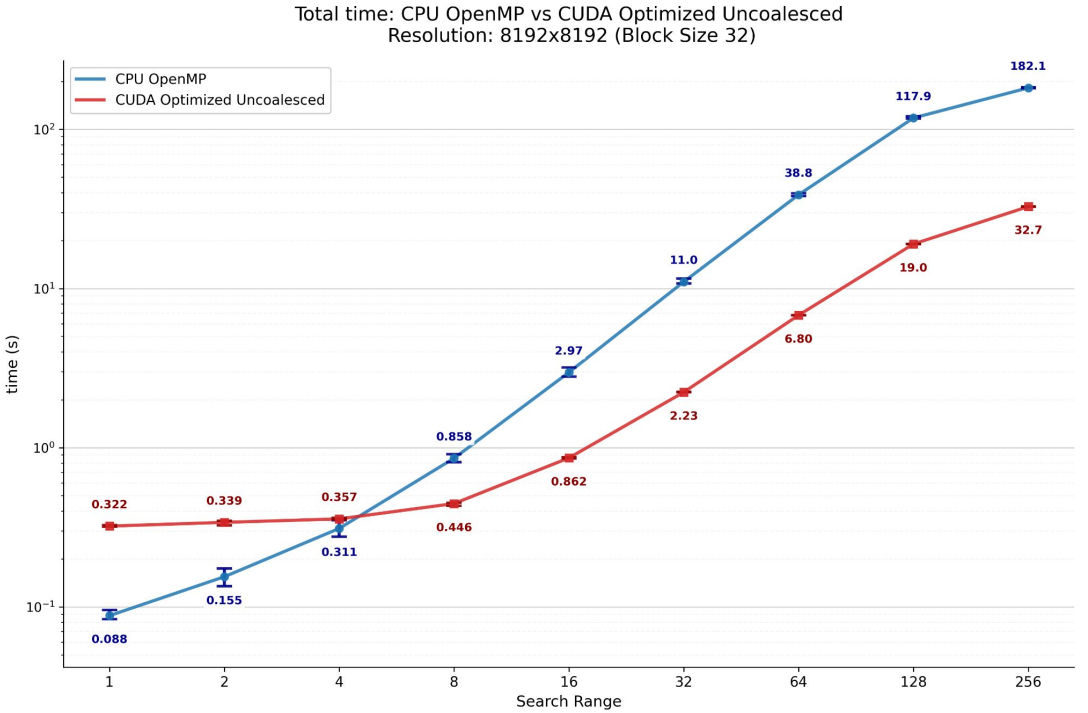
fullSearchBM_cpu_OpenMP.cpp

```
int computeSAD(const ImageGray& curr, const ImageGray& ref,
 int currX, int currY, int refX, int refY, int blockSize) {
 int sad = 0;

 // vectorize inner loop, sum partial results across SIMD lanes
 #pragma omp simd reduction(+:sad)
 for (int row = 0; row < blockSize; row++) {
  for (int col = 0; col < blockSize; col++) {
   // current block pixel
   int c_pixel = curr.data[(currY + row) * curr.width + (currX + col)];
   // candidate block pixel
   int r_pixel = ref.data[(refY + row) * ref.width + (refX + col)];
   sad += abs(c_pixel - r_pixel);
  }
 }
 return sad;
}
```

CPU OpenMP: benchmark with CUDA Optimized Uncoalesced

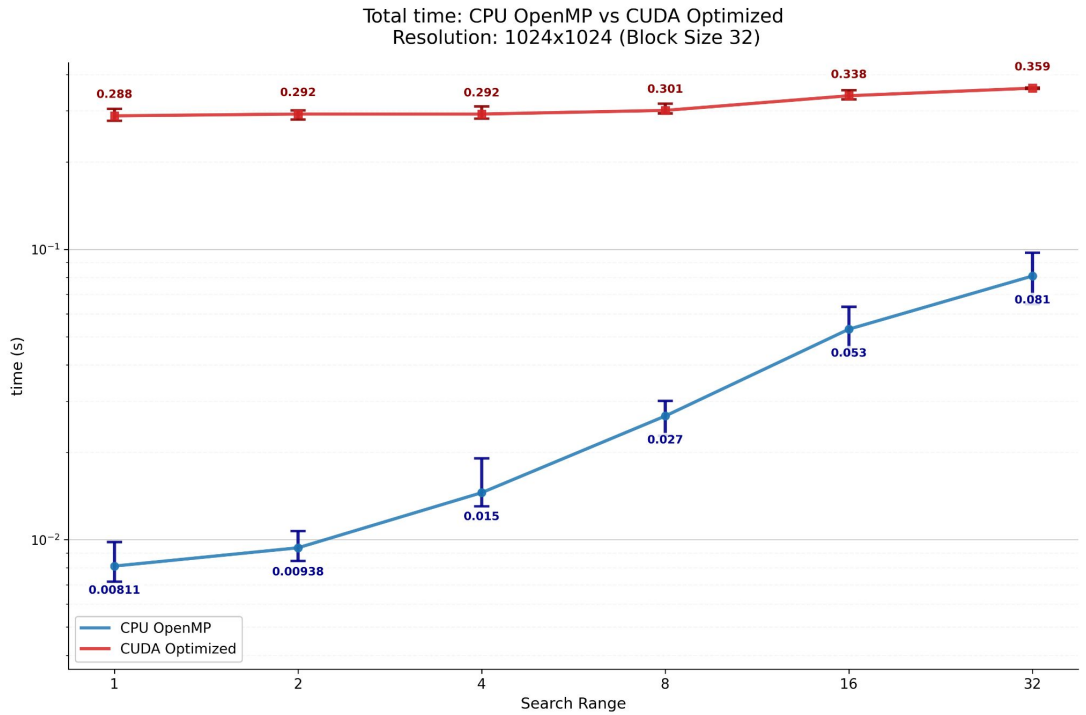
Increasing the number of blocks in the frame grid allows the GPU to better mask GMEM management overhead. This maximizes the utilization of the CUDA architecture, shifting the bottleneck to pure computation, giving better performance over the CPU OpenMP implementation.

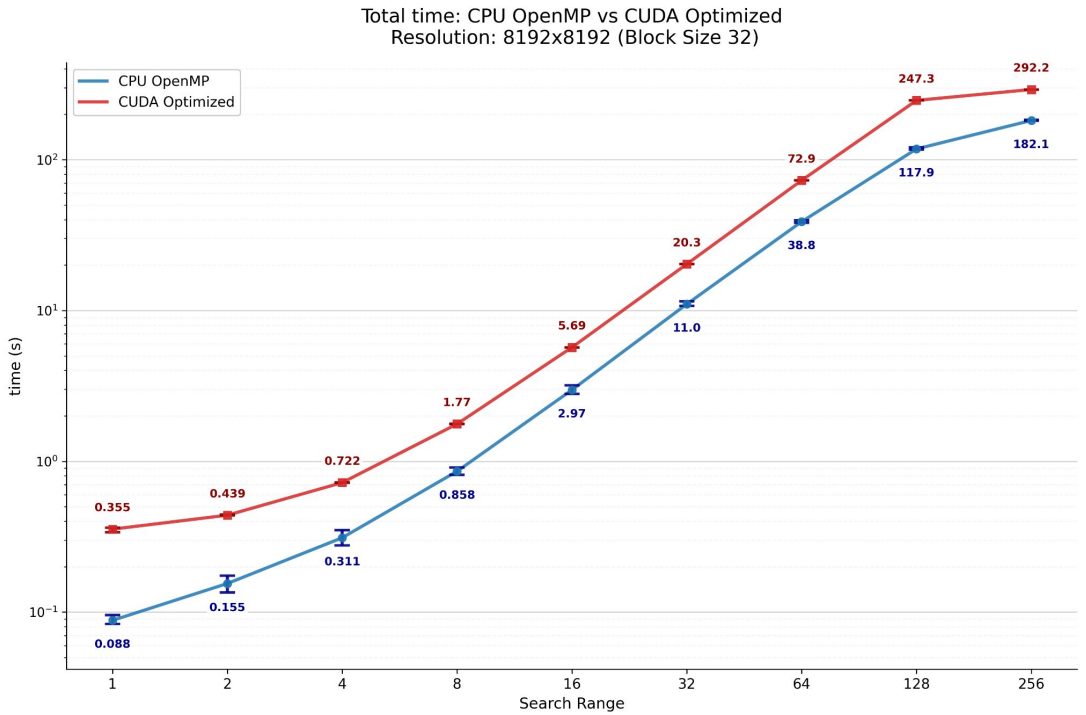




CPU OpenMP: benchmark with CUDA Optimized

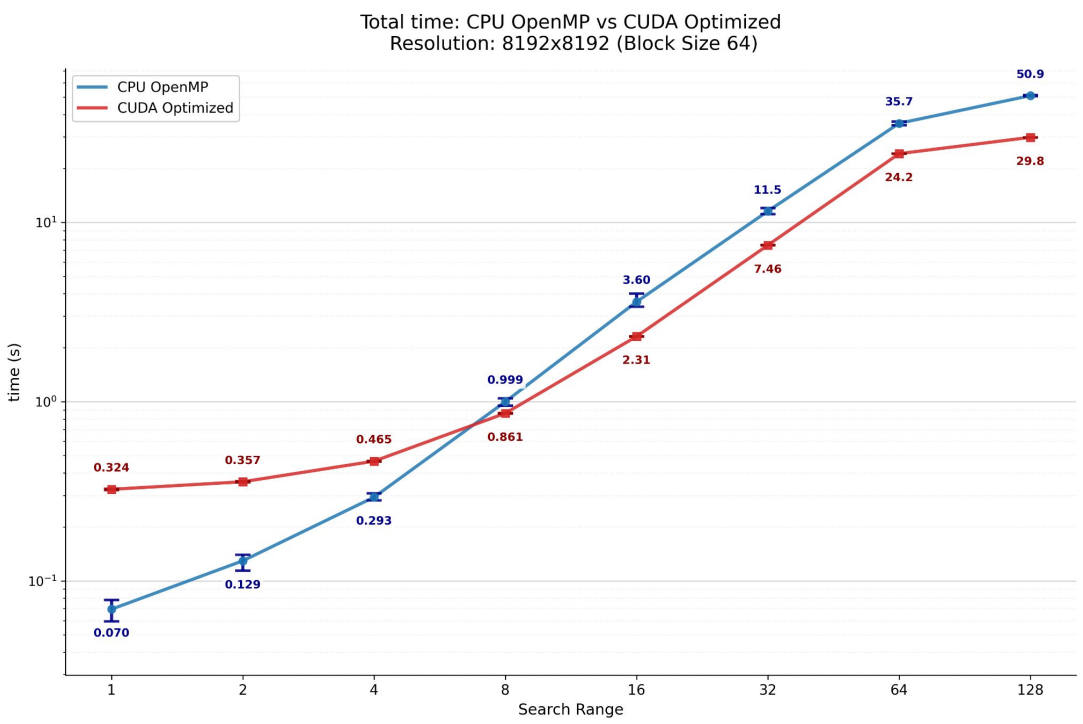
With a block size of 32, the overhead from massive *d_sad* buffer allocation, repeated GMEM accesses, and two-kernel synchronization heavily outweighs the benefits of CUDA optimized approach. Consequently, CPU OpenMP handles this specific grid density more efficiently and achieves superior performance.

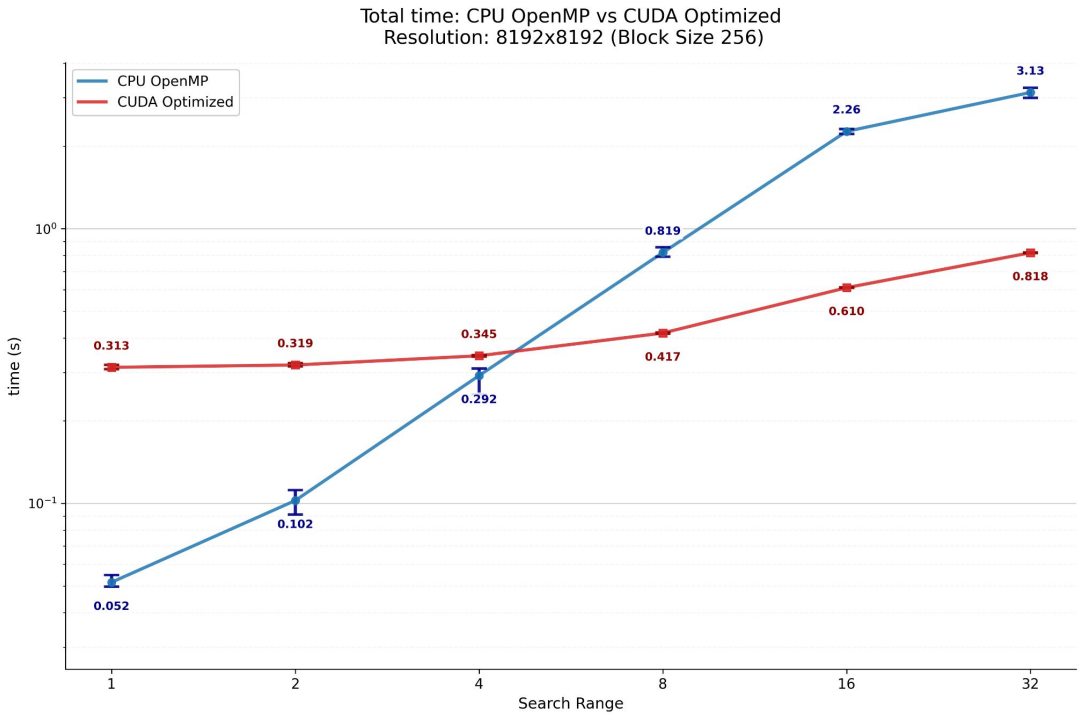




CPU OpenMP: benchmark with CUDA Optimized

Increasing the block size reduces the total number of blocks in the frame grid. This results in fewer block pairs for SAD computation, but a higher computational workload per pair. Consequently, the *d_sad* buffer shrinks, significantly reducing GMEM overhead. This shift allows the GPU to utilize its massive parallelism much more effectively than the OpenMP implementation.





Logarithmic Search

Implementations of logarithmic search algorithm

Full Search vs Logarithmic Search

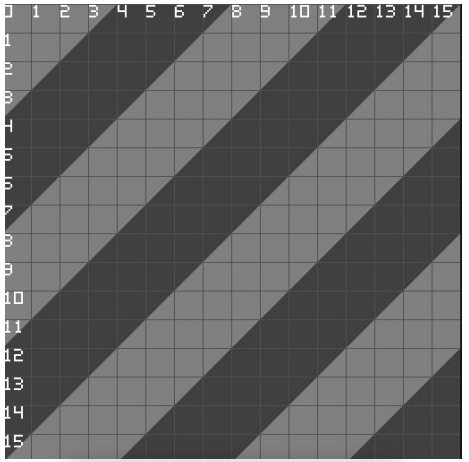
Performing a logarithmic search with a search distance defined as

$$searchDistance = max(blocksX, blocksY) - 1$$

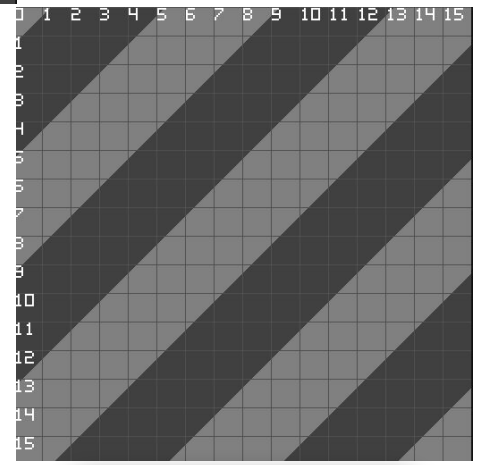
Yields an approximate result of the full search in a significantly shorter time.

While the cost to perform a full search for one frame block is computing the SAD for $blocksX \cdot blocksY$ pairs, the cost for the logarithmic search is just

$$9 \cdot log_2(searchDistance)$$



Reference Frame



Curren Frame

⚠️ **Load imbalance**

Similar to Range Search, there is a load imbalance on the border frame blocks. With this specific search distance, this behavior reaches an extreme: every single block acts as a border block, except for the one in the exact center of the frame.

Full Search vs Logarithmic Search

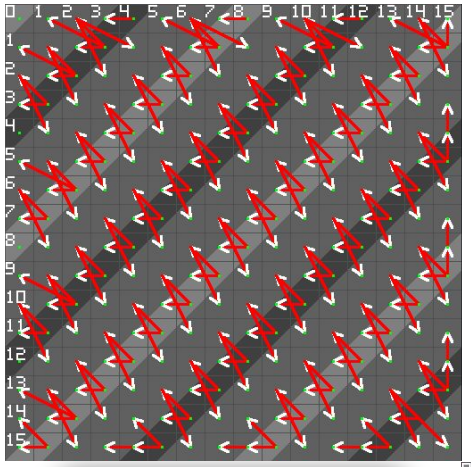
Performing a logarithmic search with a search distance defined as

$$searchDistance = max(blocksX, blocksY) - 1$$

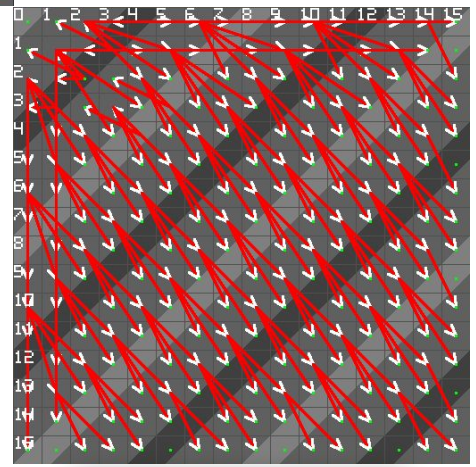
Yields an approximate result of the full search in a significantly shorter time.

While the cost to perform a full search for one frame block is computing the SAD for $blocksX \cdot blocksY$ pairs, the cost for the logarithmic search is just

$$9 \cdot log_2(searchDistance)$$



Full Search



Logarithmic Search

⚠️ **Load imbalance**

Similar to Range Search, there is a load imbalance on the border frame blocks. With this specific search distance, this behavior reaches an extreme: every single block acts as a border block, except for the one in the exact center of the frame.

CPU Naive

Sequential implementation:

1. The current frame is logically divided into frame blocks.
2. Logarithmic loop, halving the search distance at each step:
   a. Compute SAD for the search center and its 8 surrounding candidate reference frame blocks.
   b. Compute the best candidate, it becomes the new search center
3. After the final iteration, the search center corresponds to the best candidate found, which determines the motion vector for that frame block.

⚠️ **Parallelization & Execution Dependency**

Finding the motion vector for different blocks of the current frame is completely independent. However, the inner distance-halving iterations must be executed sequentially (RAW loop-carried dependency).

logarithmicSearchBM_cpu_naive.cpp

```
// Iterate over every block of the current frame
for (int bx = 0; bx < blocksX; bx++) {
 for (int by = 0; by < blocksY; by++) {
  int x = bx * blockSize, y = by * blockSize; // current block, top-left in pixels
  int cx = bx, cy = by; int bestCx = cx, bestCy = cy; // search center
  // Logarithmic search: halve distance each iteration (RAW loop-carried dependency)
  for (int d = searchDistance; d > 0; d >>= 1) {
   int bestSAD = INT_MAX, bestDist = INT_MAX;
   // Iterate over the 9 candidate positions around the search center, at distance d
   // (-d,d) (0,d) (d,d) / (-d,0) (0,0) (d,0) / (-d,-d) (0,-d) (d,-d)
   for (int ry = -d; ry <= d; ry += d) {
    for (int rx = -d; rx <= d; rx += d) {
     int cand_bx = cx + rx, cand_by = cy + ry;
     int ref_x = cand_bx * blockSize, ref_y = cand_by * blockSize;

     // check frame bounds

     // compute SAD between the current block and this candidate
     int sad = computeSAD(curr, ref, x, y, ref_x, ref_y, blockSize);
     int dist = (cand_bx - bx) * (cand_bx - bx) + (cand_by - by) * (cand_by - by);
     // tie-break on distance; remaining ties broken implicitly by scan order (ry, rx)
     if (sad < bestSAD || (sad == bestSAD && dist < bestDist)) {
      // current best SAD and MV
      bestSAD = sad; bestDist = dist; bestCx = cand_bx; bestCy = cand_by;
     }
    }
   }
   cx = bestCx; cy = bestCy;  // move search center to the best candidate
  }
  // store the final motion vector for this current frame block
  mv[bx][by] = {bestCx - bx, bestCy - by};
 }
}
```

CUDA Optimized

The algorithm employs an iterative multi-dimensional mapping, where the host halves the search distance and launches a new kernel at each step:

- **CUDA grid 3D:** the first 2 axes XY maps the frame grid. Simultaneously, the Z axis to the 9 candidate blocks in the reference.
- **CUDA block 1D**: parallelize the SAD computation.

Workflow idea: Host Side

1. Transfer (H2D) current and reference frames into GMEM.
2. Iterate, halving the search distance down to 1:
   a. Dynamic selection of the optimal caching strategy, based on available SMEM.
   b. Launch the kernel to evaluate the candidate, then reduce the results to update the best candidate for each block before proceeding to the next iteration.
3. Read final result from GMEM (D2H).

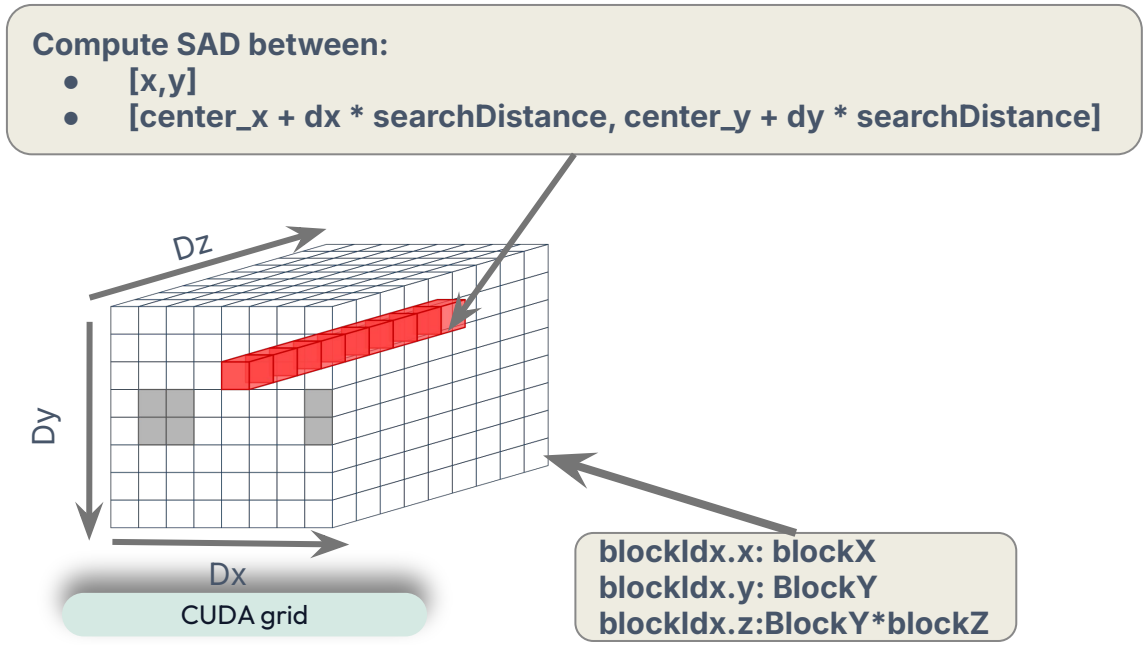
Workflow idea: Device Side (per iteration)

1. Kernel SAD:
   a. **Loading SMEM**: cooperatively load required frame blocks into SMEM.
   b. **Center update**: each block sets its search origin from the previous iteration's best match.
   c. **Intra-Candidate Reduction**: threads compute partial SADs and reduce to get the total SAD for this candidate.
2. **CUB Segmented reduce**: across the 9 candidates per block, find the best candidate,used as the search center for the next iteration, or, at the last iteration, as the final best match for that current frame block.

CUDA Optimized: grid and block dimensions

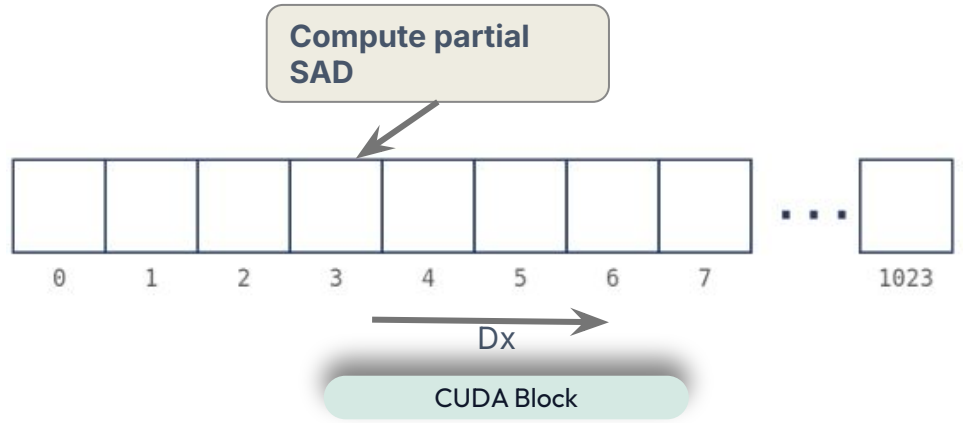
GridDim: 3 dimensions [blockX, blockY, 9]

- XY Dimensions map directly to the position of the block within the current frame.
- Z Dimension parallelize the search workload. Each block computes the SAD value between the current XY block and its assigned candidate reference frame block.



BlockDim: 1 dimension [block size*block size]

- The pixels of the current and reference frame block pair assigned to this CUDA block are divided among threads, each thread calculates the partial SAD for its assigned subset of pixels.
- The host maximizes the thread count up to the hardware limit or total pixel count, strictly enforcing a power of 2 for a safe parallel reduction.



CUDA Optimized: host

GMEM structures:

- *d_sad_candidates*:
  it stores the computed SAD value and the associated Motion Vector for each of the 9 candidate positions evaluated during the current iteration.
- *d_best_candidates: pre-initialized with a zero motion vector, so the starting search center coincides with the current frame block's own position in the reference frame; it holds the optimal MV and SAD per block, updating the center for the next step and ultimately storing the final result.*

Iterative Execution: logarithmic search loop, halving currentDistance each iteration:

- computeSADKernel: SAD for the 9 candidates per block, centered on the previous iteration's best.
- CUB DeviceSegmentedReduce: finds the best MV per block, updating d_best_candidates as the next search center.

logarithmicSearchBM_optimized.cpp: logarithmSearchCUDAOptimizedGray

```
CandidateSad* d_sad_candidates = nullptr;
CandidateSad* d_best_candidates = nullptr;
gpuErrorCheck(cudaMalloc(&d_sad_candidates,
    (size_t) blocksX * blocksY * 9 * sizeof(CandidateSad)));
gpuErrorCheck(cudaMalloc(&d_best_candidates,
    (size_t) blocksX * blocksY * sizeof(CandidateSad)));

// Initialize best candidates to max SAD
vector<CandidateSad> h_init(blocksX * blocksY, {INT_MAX, 0, 0});
cudaMemcpy(d_best_candidates, h_init.data(),
    blocksX * blocksY * sizeof(CandidateSad), cudaMemcpyHostToDevice);
```

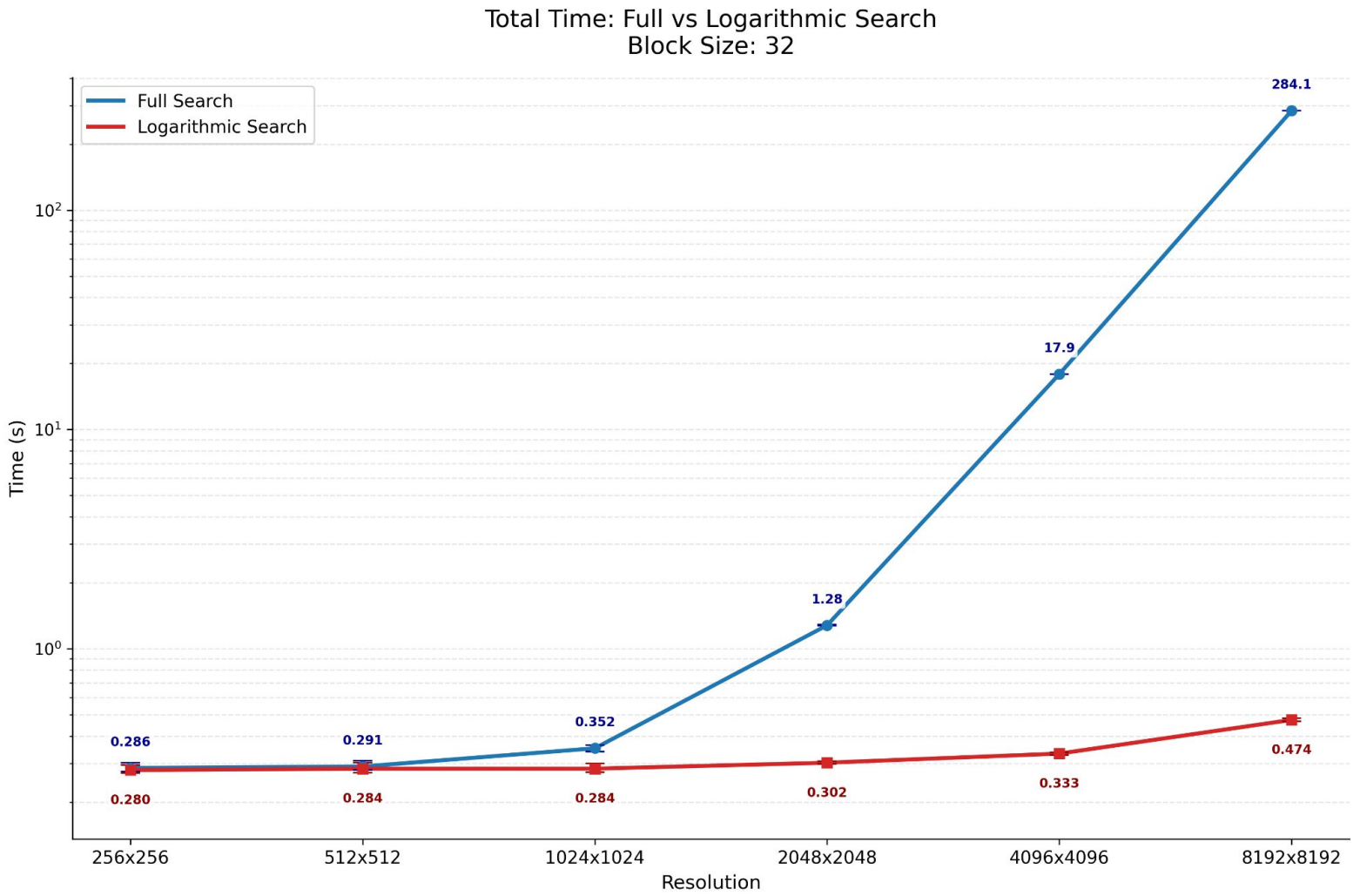
logarithmicSearchBM_optimized.cpp: logarithmSearchCUDAOptimizedGray

```
for (int currentDistance = searchDistance; currentDistance > 0;
currentDistance >>= 1) {
 // Kernel SAD: compute SAD for the 9 candidate positions at this dist
 launchSADKernel(deviceProp, smOneFrame, smTwoFrames, gridKSad,
  threadsKSad, strategy, smKSad, d_curr, d_ref, curr.width,
  d_sad_candidates, d_best_candidates, blockSize, blocksX, blocksY,
  currentDistance);
 // CUB Segmented Reduce: find the best candidate among the 9, update
  center for next iteration
 cub::DeviceSegmentedReduce::Reduce(d_temp_storage, temp_storage_bytes,
  d_sad_candidates, d_best_candidates, frameBlocks, d_offsets,
  d_offsets + 1, MotionVectorUtils::CandidateSadOp(),
  CandidateSad{INT_MAX, 0, 0});
}
```

CUDA Optimized: benchmark

At smaller frame grid, the performance gap shrinks. Computing fewer candidates is offset by the overhead of launching multiple consecutive kernels and frequent CPU-GPU synchronizations per search step.

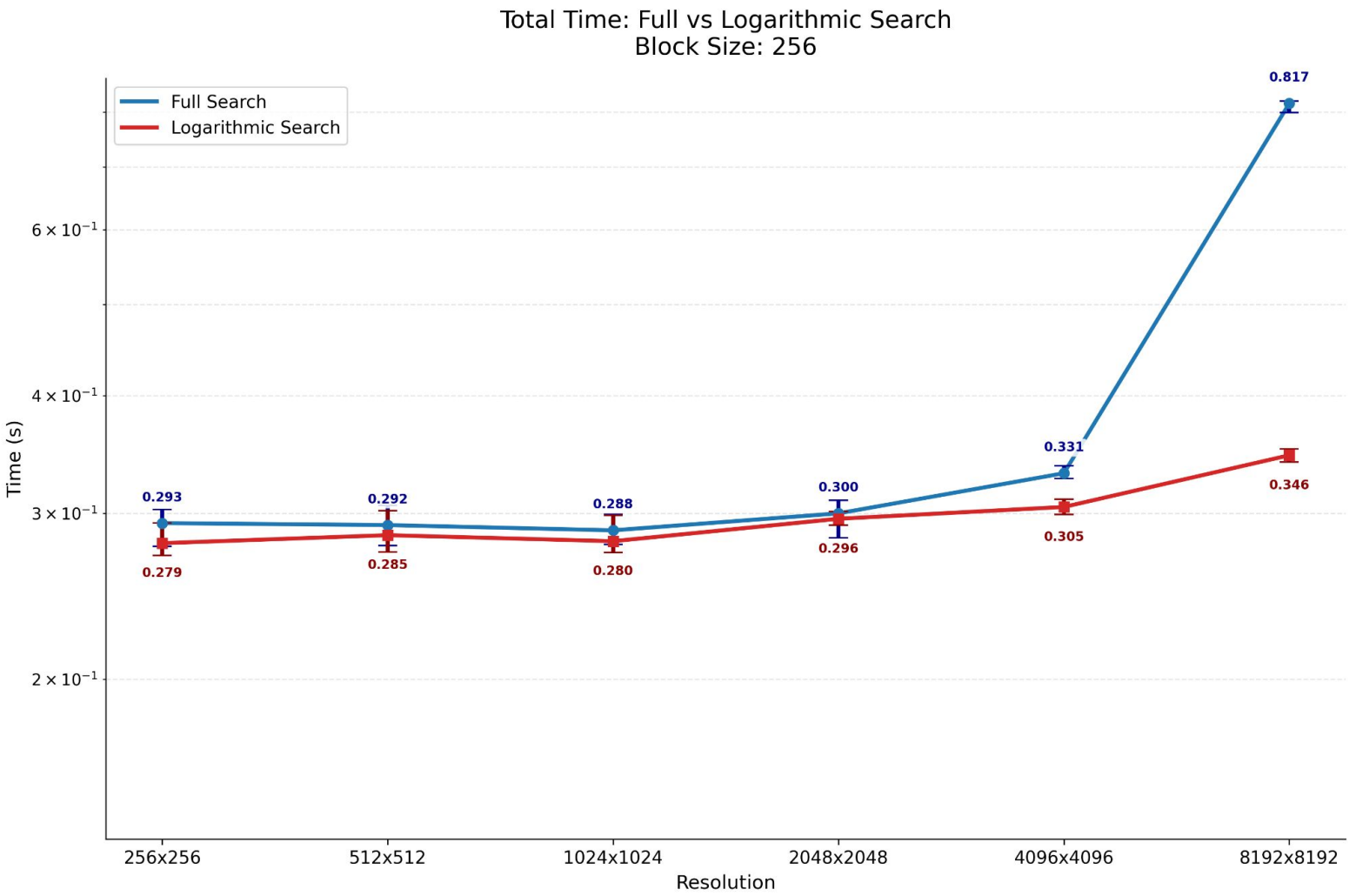
Block size (resolution 4096 × 4096)	CUDA grid and block *ComputeSADKernel*	Number of candidate per current frame block
32	grid: 128 × 128 × 9 block: 1024 × 1 × 1	63
64	grid: 64 × 64 × 9 block: 1024 × 1 × 1	54
128	grid: 32 × 32 × 9 block: 1024 × 1 × 1	45
256	grid: 16 × 16 × 9 block: 1024 × 1 × 1	36



CUDA Optimized: benchmark

At smaller frame grid, the performance gap shrinks. Computing fewer candidates is offset by the overhead of launching multiple consecutive kernels and frequent CPU-GPU synchronizations per search step.

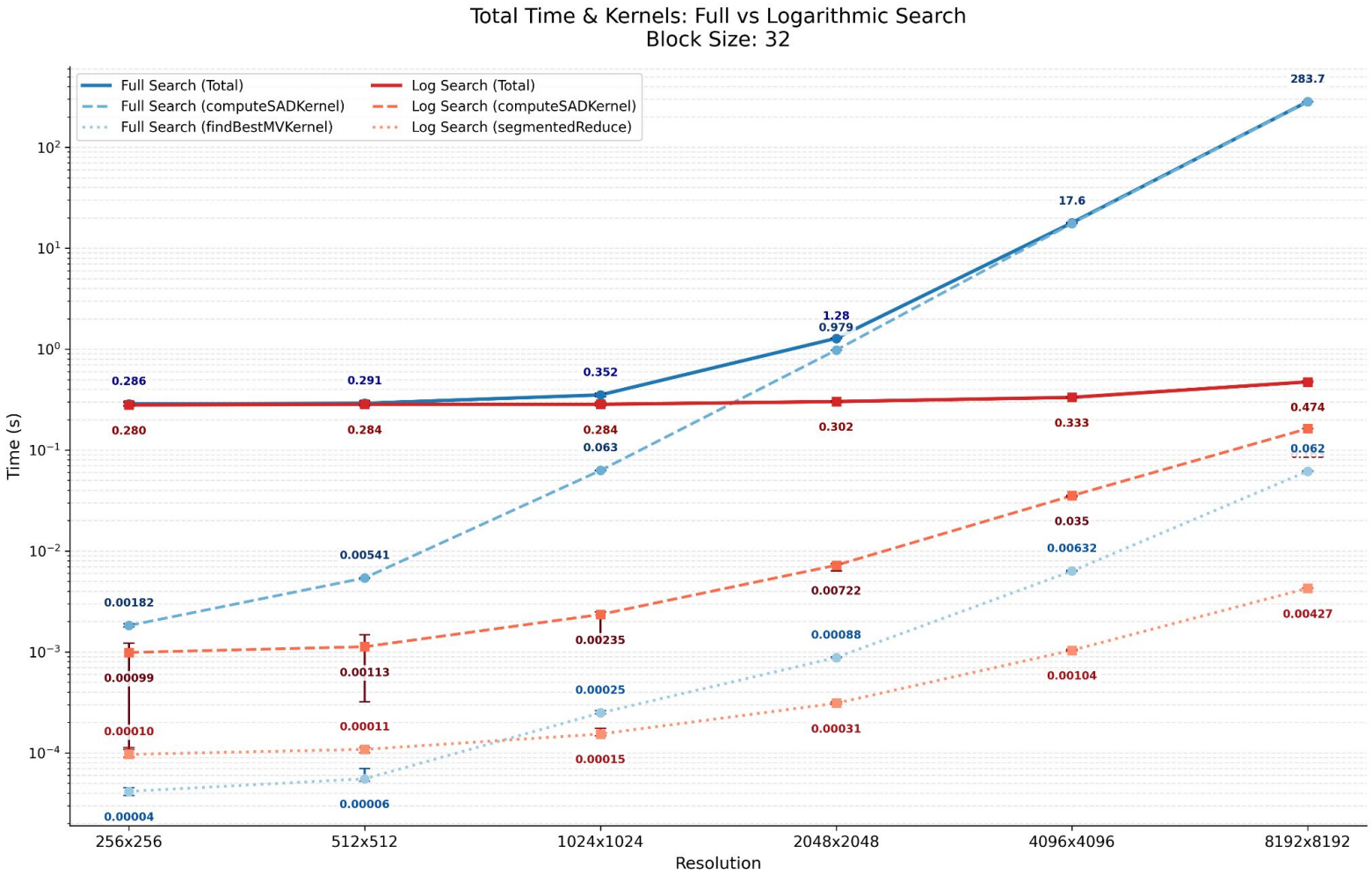
Block size (resolution 4096 × 4096)	GMEM Logarithmic Search	GMEM Full Search
32	d_sad_candidates: 1728 KiB d_best_candidates: 192 KiB	d_sad: 1024 MiB
64	d_sad_candidates: 432 KiB d_best_candidates: 48 KiB	d_sad: 6 MiB
128	d_sad_candidates: 108 KiB d_best_candidates: 12 KiB	d_sad: 4 MiB
256	d_sad_candidates: 27 KiB d_best_candidates: 3 KiB	d_sad: 256 KiB



CUDA Optimized: benchmark

As the grid shrinks, the synchronization overhead of repeatedly launching *segmentedReduce* overtakes the actual findBestMVKernel computation.

Block size (resolution 4096 × 4096)	CUDA grid and block *ComputeSADKernel*	Number of candidate per current frame block
32	grid: 128 × 128 × 9 block: 1024 × 1 × 1	63
64	grid: 64 × 64 × 9 block: 1024 × 1 × 1	54
128	grid: 32 × 32 × 9 block: 1024 × 1 × 1	45
256	grid: 16 × 16 × 9 block: 1024 × 1 × 1	36



Questions?

Thanks for the attention.